OPERATING SYSTEMS
ASSIGNMENT – CO3
CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and
segmentation to determine their effectiveness for different system requirements – linking and
loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques
and disk management.
Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date:
Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)
Mandatory Execution and GitHub Submission
• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
• Create a GitHub repository named CSA04-<RegisterNumber>-Assignment3 and share
it with the course faculty (public, or private with the faculty added as a collaborator).
• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing
the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs
and screenshots.
• Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
• Commit your work progressively – at least one commit for each task. The commit history
will be checked; a single upload of all files on the last day will not be accepted.
• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.
GitHub Repository Link: https://github.com/192524372simats/CSA0413-OS.git
How to Answer the Additional Questions
Complete the Code: Copy the given code into your repository, replace every TODO with
working code without changing the rest of the program, execute it and compare your output (or
result) with the expected output.
Alternative Concept and Complexity Analysis: Describe how the solution would change if the
alternative concept were used, with code or pseudocode, and analyse and compare the time

complexity and space complexity of both solutions using Big-O notation, supported by your own
measurements wherever possible.

Assignment Principle
Every question asks you to design, implement, analyse and evaluate the solution yourself, so
that you demonstrate evidence of:
Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding (6 marks)

1.1 Problem and design

The bank's branch software is built from many modules. If the encryption library is linked **statically**, every security fix means rebuilding and shipping the whole executable to every branch (200 MB in the scenario). If it is linked **dynamically**, only the small shared library has to be replaced. To test this I built a toy 'bank' program (`bank.c`) that calls an encryption library (`enc.c`, a stand-in for the real crypto library) and built it three ways: static, dynamic, and with `dlopen()`.

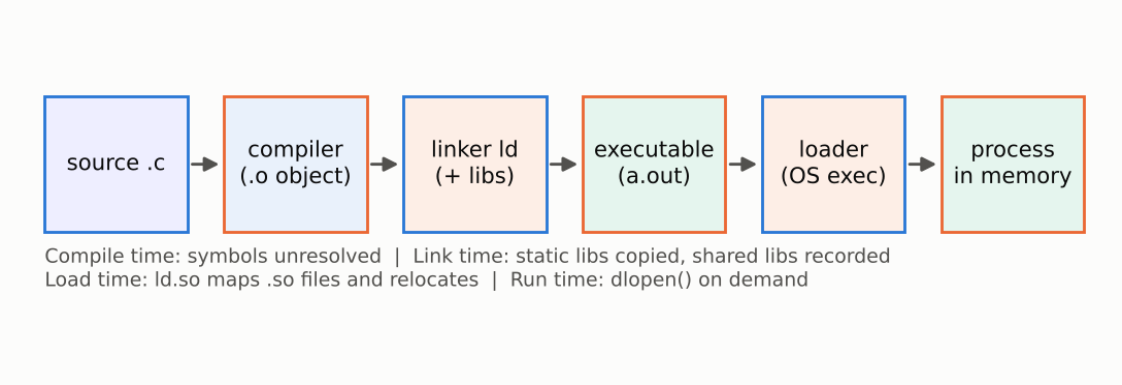


Figure 1.1 – From source to running process: where linking, loading and binding happen

1.2 Implementation

enc.c – the library (version 1)

```
/* "encryption library" v1: simple Caesar-style cipher (stand-in for the bank's crypto lib)
*/
#include <string.h>
const char *enc_version(void){ return "libenc v1 (shift cipher)"; }
void encrypt(const char *in, char *out){
    size_t n = strlen(in);
    for(size_t i=0;i<n;i++) out[i] = in[i] + 3;
    out[n]=0;
}
```

bank.c – the application

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "enc.h"
int main(int argc,char**argv){
    char out[128];
    encrypt("ACCT-1234-BALANCE", out);
    printf("%s -> ", enc_version());
    for(char*p=out;*p;p++) printf("%02x", (unsigned char)*p);
```

```
    printf("\n");
    if(argc>1 && argv[1][0]=='m'){          /* print own memory map */
        FILE*f=fopen("/proc/self/maps","r"); char l[512];
        while(fgets(l,sizeof l,f)) fputs(l,stdout);
        fclose(f);
    }
    if(argc>1 && argv[1][0]=='s') sleep(3);  /* stay alive for smaps inspection */
    if(argc>1 && argv[1][0]=='r'){          /* print own memory use */
        FILE*f=fopen("/proc/self/status","r"); char l[256];
        while(fgets(l,sizeof l,f)) if(!strncmp(l,"VmRSS",5)||!strncmp(l,"VmSize",6))
fputs(l,stdout);
        fclose(f);
    }
    return 0;
}
```

lazy.c – loads the library at run time with dlopen()/dlsym()

```
#include <string.h>
/* Alternative concept: load the library with dlopen() only when needed */
#include <stdio.h>
#include <dlfcn.h>
int main(int argc,char**argv){
    int need = argc>1;                      /* any argument = encryption needed */
    printf("program started, encryption needed? %s\n", need?"yes":"no");
    if(!need){ printf("no library loaded - nothing to fail\n"); return 0; }
    const char*path = (argc>2)?argv[2]:"./libenc.so";
    void*h = dlopen(path, RTLD_NOW);
    if(!h){ fprintf(stderr,"dlopen failed: %s\n(falling back: refusing to process, program
keeps running)\n", dlerror()); return 2; }
    void (*enc)(const char*,char*) = (void(*)(const char*,char*))dlsym(h,"encrypt");
    const char*(*ver)(void) = (const char*(*)(void))dlsym(h,"enc_version");
    if(!enc||!ver){ fprintf(stderr,"dlsym failed: %s\n",dlerror()); return 3; }
    char out[128]; enc("ACCT-1234-BALANCE",out);
    printf("%s -> ",ver()); for(char*p=out;*p;p++) printf("%02x",(unsigned char)*p);
puts("");
    FILE*f=fopen("/proc/self/maps","r"); char l[512];
    printf("--- maps lines containing libenc ---\n");
    while(fgets(l,sizeof l,f)) if(strstr(l,"libenc")) fputs(l,stdout);
    fclose(f); dlclose(h); return 0;
}
```

Build commands

```
gcc -std=c11 -Wall -Wextra -fPIC -shared -o libenc.so enc.c                    # shared
library (position-independent)
gcc -std=c11 -Wall -Wextra -o bank_dyn bank.c -L. -lenc -Wl,-rpath,'$ORIGIN'
# dynamic build
```

```
gcc -std=c11 -Wall -Wextra -static -o bank_static bank.c enc.c                  # static build
gcc -std=c11 -Wall -Wextra -o lazy lazy.c -ldl                                  # dlopen build
```

Sizes and shared libraries (ldd)

Output

```
$ ls -l bank_dyn bank_static libenc.so
-rwxr-xr-x 1 root root  16376 Nov 11 04:52 bank_dyn
-rwxr-xr-x 1 root root 785864 Nov 11 04:52 bank_static
-rwxr-xr-x 1 root root  15584 Nov 11 04:52 libenc.so

$ size bank_dyn bank_static
   text       data        bss        dec        hex    filename
   2683        704         16       3403        d4b    bank_dyn
 670535      23376      22440     716351      aee3f    bank_static

$ ldd bank_dyn
    linux-vdso.so.1 (0x00007f653dcde000)
    libenc.so => /home/claude/os/Q1/./libenc.so (0x00007f653dccc000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f653da00000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f653dce0000)

$ ldd bank_static
    not a dynamic executable

$ ./bank_dyn ; ./bank_static
libenc v1 (shift cipher) -> 4446465730343536373045444f44514648
libenc v1 (shift cipher) -> 4446465730343536373045444f44514648
```

Build	Executable size	Must also be shipped on update
Static	785,864 bytes (≈ 768 KB, includes the whole libc)	the whole executable
Dynamic	16,376 bytes	only libenc.so (15,584 bytes)

The static build is **48× larger** than the dynamic one even for this toy program, because the whole C library is copied into it. ldd shows the dynamic build needs libenc.so, libc.so.6 and the loader ld-linux-x86-64.so.2; ldd bank_static reports ‘not a dynamic executable’. With the bank's real 200 MB program, the same effect means a 15–20 MB library patch instead of 200 MB per branch.

Where are program and libraries loaded? (/proc/self/maps)

Output (excerpt – the program prints /proc/self/maps)

```
$ ./bank_dyn m   (maps of dynamic build)
libenc v1 (shift cipher) -> 444646573034353637304544f44514648
561d0e3e8000-561d0e3e9000 r--p 00000000 fe:00 795150
/home/claude/os/Q1/bank_dyn
561d0e3e9000-561d0e3ea000 r-xp 00001000 fe:00 795150
/home/claude/os/Q1/bank_dyn
561d0e3ea000-561d0e3eb000 r--p 00002000 fe:00 795150
/home/claude/os/Q1/bank_dyn
561d0e3eb000-561d0e3ec000 r--p 00002000 fe:00 795150
/home/claude/os/Q1/bank_dyn
561d0e3ec000-561d0e3ed000 rw-p 00003000 fe:00 795150
/home/claude/os/Q1/bank_dyn
7f3e22000000-7f3e22028000 r--p 00000000 fe:00 320311
/usr/lib/x86_64-linux-gnu/libc.so.6
7f3e22028000-7f3e221b1000 r-xp 00028000 fe:00 320311
/usr/lib/x86_64-linux-gnu/libc.so.6
7f3e221b1000-7f3e22200000 r--p 001b1000 fe:00 320311
/usr/lib/x86_64-linux-gnu/libc.so.6
7f3e22200000-7f3e22204000 r--p 001ff000 fe:00 320311                  /usr/lib/x86_64-
linux-gnu/libc.so.6
7f3e22204000-7f3e22206000 rw-p 00203000 fe:00 320311
/usr/lib/x86_64-linux-gnu/libc.so.6
7f3e2240b000-7f3e2240c000 r--p 00000000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f3e2240c000-7f3e2240d000 r-xp 00001000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f3e2240d000-7f3e2240e000 r--p 00002000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f3e2240e000-7f3e2240f000 r--p 00002000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f3e2240f000-7f3e22410000 rw-p 00003000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f3e22418000-7f3e2241a000 r-xp 00000000 00:00 0                   [vdso]
7f3e2241a000-7f3e2241b000 r--p 00000000 fe:00 320291
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f3e2241b000-7f3e22446000 r-xp 00001000 fe:00 320291
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f3e22446000-7f3e22450000 r--p 0002c000 fe:00 320291
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f3e22450000-7f3e22452000 r--p 00036000 fe:00 320291
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f3e22452000-7f3e22454000 rw-p 00038000 fe:00 320291
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe76e65000-7ffe76e8b000 rw-p 00000000 00:00 0                   [stack]
```

```
$ ./bank_dyn m   (second run - note addresses change: ASLR)
libenc v1 (shift cipher) -> 444646573034353637304544f44514648
55f22de66000-55f22de67000 r--p 00000000 fe:00 795150
/home/claude/os/Q1/bank_dyn
55f22de67000-55f22de68000 r-xp 00001000 fe:00 795150
/home/claude/os/Q1/bank_dyn

$ ./lazy y ./libenc.so (dlopen/dlsym)
program started, encryption needed? yes
libenc v1 (shift cipher) -> 444646573034353637304544f44514648
--- maps lines containing libenc ---
7f6d1bcc9000-7f6d1bcca000 r--p 00000000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f6d1bcca000-7f6d1bccb000 r-xp 00001000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f6d1bccb000-7f6d1bccc000 r--p 00002000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f6d1bccc000-7f6d1bccd000 r--p 00002000 fe:00 795151
/home/claude/os/Q1/libenc.so
7f6d1bccd000-7f6d1bcce000 rw-p 00003000 fe:00 795151
/home/claude/os/Q1/libenc.so

$ ./lazy        (no encryption needed)
program started, encryption needed? no
... (2 more lines)
```

The program (`bank_dyn`) is mapped at `0x561d0e3e8000`, libc at `0x7f3e22000000` and `libenc.so` at `0x7f3e2240b000`. A second run placed the program at `0x55f22de66000`: the addresses change on every run because of **ASLR** (address-space layout randomisation). Each library appears as several mappings with different permissions: `r--p` (headers), `r-xp` (code), `r--p` (read-only data / relro) and `rw-p` (writable data) — the code is read-only and executable, the data is writable but not executable.

1.3 Evaluation

Address binding – compile, load and execution time

Binding time	Meaning	Example	Consequence
Compile time	Compiler/assembler knows the absolute memory location	MS-DOS `.COM` programs, embedded firmware	Program must always load at the same address; recompile to move it

Binding time	Meaning	Example	Consequence
Load time	Compiler produces relocatable code; the loader fixes addresses when the program is loaded	Old a.out/ELF non-PIC code, relocations applied by ld.so	Cannot move after start; relocation costs time at start-up
Execution time	Binding is delayed until run; hardware (MMU) translates every logical address	Paging/segmentation in all modern OSs; ASLR; dlopen()	Process can be moved/paged out; needs MMU support

Roles of the linker and the loader

- **Linker (`ld`)** – combines object files and libraries, resolves symbols (e.g. encrypt) to addresses or to import records, and creates relocation tables. Static linking copies library code in; dynamic linking only records the library name (DT_NEEDED) and the needed symbols.
- **Loader (kernel `execve` + `ld.so`)** – creates the process, maps the segments of the executable (the maps above), maps each shared library, applies the relocations, runs initialisers and jumps to main. dlopen() asks the same loader to do this work at run time.

Static versus dynamic linking

Criterion	Static	Dynamic (shared)	Measured here
Disk / network size	Large (library copied into every program)	Small program + one shared library	785,864 B vs 16,376 B
Memory use (many processes)	Each process has its own copy of the code	Library code pages shared by all processes	see experiment 1.4 – same physical page
Memory of one process	Smaller (only the needed code)	Larger resident set (libc + loader mapped)	VmRSS 748 kB vs 1,696 kB
Start-up time	Fastest – nothing to resolve	Slower – loader maps libraries and relocates	0.96 ms vs 1.27 ms (+33 %)
Ease of update / security patch	Rebuild and redeploy every program	Replace one .so, no recompilation	see 1.4 – md5 of bank_dyn unchanged

Criterion	Static	Dynamic (shared)	Measured here
Dependency risk	None – self-contained	Missing/incompatible library breaks the program	exit code 127 when missing

Position-independent code (PIC)

PIC is machine code that works correctly wherever it is loaded, because it never contains absolute addresses. Instructions use **PC-relative addressing** for code and data inside the library, and access global data and external functions **indirectly through the GOT (Global Offset Table) and PLT (Procedure Linkage Table)**, which the loader fills in. Because the code pages are never modified by relocation they stay read-only and can be shared by all processes, and each process can map the library at a different address (ASLR). The check below shows that our library has no text relocations (TEXTREL), i.e. its code is truly position independent:

Output

```
$ readelf -d libenc.so | grep -E "TEXTREL|FLAGS"
(no TEXTREL -> position independent)
$ objdump -d libenc.so | grep -A3 "<enc_version>:"
0000000000001119 <enc_version>:
    1119:    f3 0f 1e fa             endbr64
    111d:    55                      push   %rbp
    111e:    48 89 e5                mov    %rsp,%rbp
```

1.4 Experimental investigation

(a) Start-up time and memory use

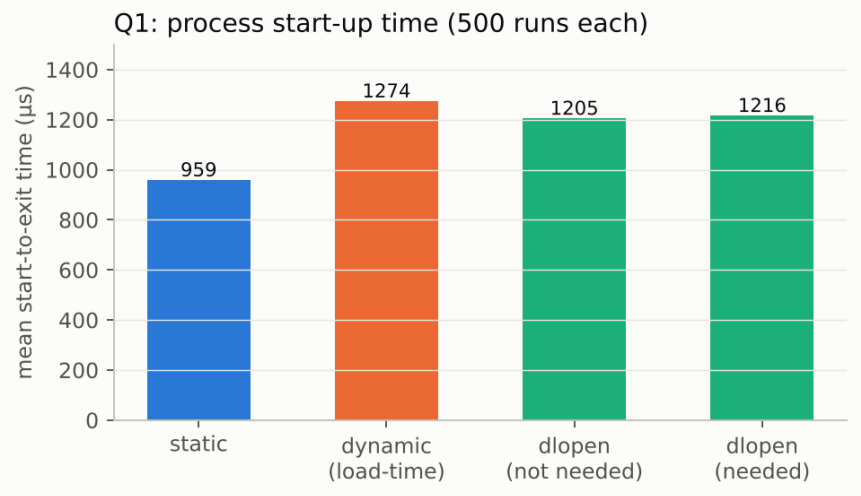


Figure 1.2 – Mean process start-up time, 500 runs of each program

Output

```
static                mean    958.9 us  median    881.7 us
dynamic                  mean   1273.8 us  median   1245.2 us
```

```
dlopen-lazy (not needed)     mean   1204.7 us  median   1243.5 us
dlopen-lazy (needed)         mean   1215.9 us  median   1127.7 us
```

Memory of one process (VmRSS/VmSize from /proc/self/status)

```
$ ./bank_static r
libenc v1 (shift cipher) -> 4446465730343536373045444f44514648
VmSize:        1036 kB
VmRSS:          748 kB
$ ./bank_dyn r
libenc v1 (shift cipher) -> 4446465730343536373045444f44514648
VmSize:        2732 kB
VmRSS:         1696 kB
```

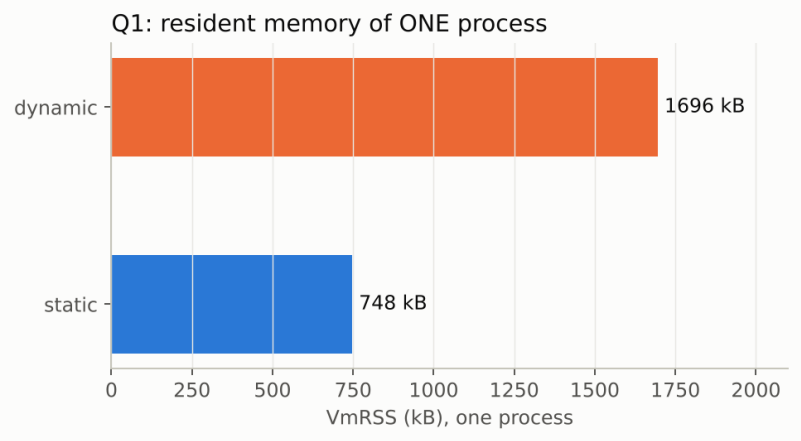


Figure 1.3 – Resident memory of one process

The dynamic version starts about **0.31 ms (≈ 33 %) later** because the loader has to find, map and relocate `libc.so.6` and `libenc.so`. One dynamic process has a larger resident set (1,696 kB vs 748 kB) because libc and the loader are mapped — but those pages belong to files that are **shared** with every other process (next experiment), so with many processes the dynamic system uses much less memory in total.

(b) Replacing the shared library without recompiling

The security fix (`enc_v2.c`, a different cipher) was compiled into a new `libenc.so` and copied over the old one. The program was not touched — its md5 checksum is the same before and after — yet its output changed. The static program kept the old, unpatched behaviour:

Output

```
=== Replace library WITHOUT recompiling bank_dyn ===
$ ./bank_dyn   (before)
libenc v1 (shift cipher) -> 4446465730343536373045444f44514648
$ gcc -fPIC -shared -o libenc.so enc_v2.c   # only the library rebuilt
$ ./bank_dyn   (after - same binary, md5 unchanged)
libenc v2 (SECURITY FIX: xor cipher) -> 1b19190e776b68696e77181b161b14191f
e01b69087e7a5262039f8115db8b68b9  bank_dyn
```

```
$ ./bank_static (still old code - needs full rebuild + redeploy)
libenc v1 (shift cipher) -> 44464657303435363730454 44f44514648

=== Missing library ===
$ ./bank_dyn
./bank_dyn: error while loading shared libraries: libenc.so: cannot open shared object
file: No such file or directory
exit code 127
$ ./lazy y ./libenc.so
dlopen failed: ./libenc.so: cannot open shared object file: No such file or directory
(falling back: refusing to process, program keeps running)
program started, encryption needed? yes
exit code 2
$ ./bank_static (unaffected)
libenc v1 (shift cipher) -> 44464657303435363730454 44f44514648
```

(c) Two copies running – which pages are shared?

Two instances of `bank_dyn` were kept alive and `/proc/<pid>/smaps` and `/proc/<pid>/pagemap` were read. The code page of `libenc.so` is at different virtual addresses in the two processes (`0x7f54d47b6000` and `0x7f5ab902a000`) but **maps to the same physical frame number (PFN 1192303)** — the page is physically stored once. For libc, 1,496 kB of its 1,520 kB resident pages are `Shared_Clean`.

Output

```
=== Two running copies: smaps of each ===
PID 2262
  /home/claude/os/Q1/bank_dyn Private_Clean: 0 kB
  /home/claude/os/Q1/bank_dyn Private_Dirty: 8 kB
  /home/claude/os/Q1/bank_dyn Rss: 20 kB
  /home/claude/os/Q1/bank_dyn Shared_Clean: 0 kB
  /home/claude/os/Q1/libenc.so Private_Clean: 0 kB
  /home/claude/os/Q1/libenc.so Private_Dirty: 8 kB
  /home/claude/os/Q1/libenc.so Rss: 20 kB
  /home/claude/os/Q1/libenc.so Shared_Clean: 0 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Private_Clean: 0 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Private_Dirty: 24 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Rss: 1520 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Shared_Clean: 1496 kB
PID 2263
  /home/claude/os/Q1/bank_dyn Private_Clean: 0 kB
  /home/claude/os/Q1/bank_dyn Private_Dirty: 8 kB
  /home/claude/os/Q1/bank_dyn Rss: 20 kB
  /home/claude/os/Q1/bank_dyn Shared_Clean: 0 kB
  /home/claude/os/Q1/libenc.so Private_Clean: 0 kB
  /home/claude/os/Q1/libenc.so Private_Dirty: 8 kB
  /home/claude/os/Q1/libenc.so Rss: 20 kB
```

```
  /home/claude/os/Q1/libenc.so Shared_Clean: 0 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Private_Clean: 0 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Private_Dirty: 24 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Rss: 1520 kB
  /usr/lib/x86_64-linux-gnu/libc.so.6 Shared_Clean: 1496 kB
pid 2262 libenc code page: virtual->PFN {'0x7f54d47b6000': 1192303}
pid 2263 libenc code page: virtual->PFN {'0x7f5ab902a000': 1192303}
```

(d) Missing shared library

With the library removed the dynamically linked program refuses to start (`error while loading shared libraries`, exit code 127). The statically linked program is unaffected. The `dlopen()` program receives `NULL` from `dlopen()`, prints the reason from `dlerror()` and exits cleanly with its own error code — the failure can be handled by the program (see 1.6).

1.5 Conclusion – which linking method should the bank use?

The bank should use **dynamic (shared) linking** for the encryption library and other common libraries. Security fixes are the main reason: replacing a ~15 KB library is far cheaper, faster and less error-prone than rebuilding and distributing a 200 MB executable to every branch, and all programs on a machine get the fix at once. Shared code also saves RAM when many programs run. The costs — about 0.3 ms slower start-up and the risk of a missing or incompatible library — are small for a banking application that runs for hours, and can be controlled by package management with versioned library names (`libenc.so.1`), checksum/signature verification of the library, and a health check at start-up. Static linking is only justified for small recovery or rescue tools that must run even when the system libraries are damaged.

1.6 Alternative concept – load the library with dlopen() only when needed

Pseudocode (full program: lazy.c above)

```
void *h = dlopen("libenc.so.1", RTLD_NOW);          /* only when encryption is first
needed */
if (!h) { log(dlerror()); return ERR_NO_CRYPTO; }     /* fail CLOSED - never send
plaintext */
encrypt_fn enc = dlsym(h, "encrypt");
if (!enc) { log(dlerror()); dlclose(h); return ERR_BAD_LIB; }
enc(in, out);   /* ... later optionally dlclose(h) */
```

Criterion	Linked at start-up (load-time)	dlopen() when needed	Measured / analysed
Start-up time	loader must map and relocate the library first	not paid until first use	1,273.8 µs (dynamic) vs 1,204.7 µs (dlopen, not needed) – the ≈ 70 µs difference is the cost of loading libenc.so; 1,215.9 µs when it is needed
Memory when encryption is not needed	library is mapped (virtual) and its touched pages resident	nothing mapped at all	0 vs O(L) pages, L = library size
Missing library	process fails at start with exit 127 – nothing runs	program starts; only the encryption feature fails and the program can decide what to do	handled via dlopen()==NULL
Error handling needed	none in the program (loader aborts)	must check dlopen, dlsym, version/ABI and fail closed; must not fall back to plaintext	extra code, more testing
Call cost	through PLT/GOT	through a function pointer – the same cost	identical

Complexity. Load-time linking costs O(S + R) once at start-up (S = symbols to resolve, R = relocations in the library) and O(L) memory for the whole run. dlopen() pays the same O(S + R) but **only if and when** the feature is used, and 0 memory otherwise; each later call is O(1) in both cases. The price is a larger error-handling surface: a missing library becomes a run-time condition that the application must handle. For the bank I would keep normal dynamic linking for the core library (a missing crypto library must stop the program anyway) and use dlopen() for rarely used optional modules such as reporting or alternative cipher plug-ins.

Q2 – Paging (8 marks)

2.1 External fragmentation with contiguous allocation

With contiguous allocation each process needs one unbroken block. As processes finish, the free memory is split into holes. In Figure 2.1 the server has 30 MB free in five holes, but the largest hole is only 8 MB, so a 20 MB process cannot be loaded. This is **external fragmentation**: enough total memory is free, but not contiguously.

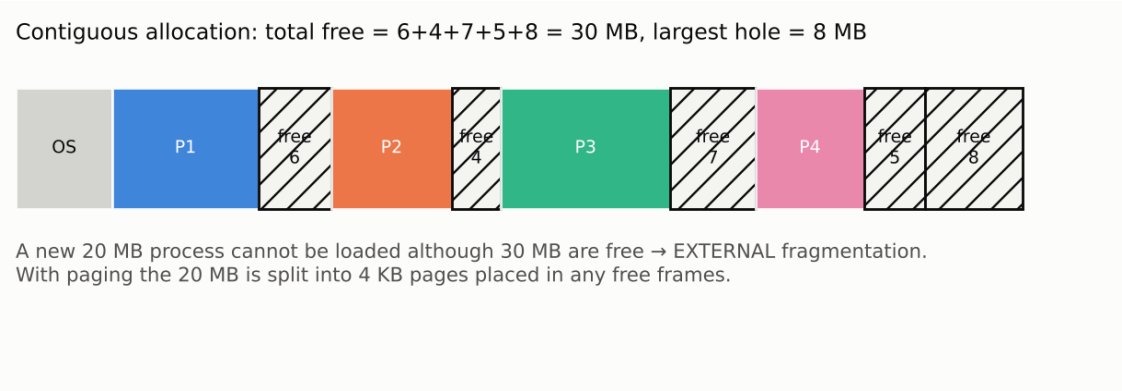


Figure 2.1 – External fragmentation (hatched = free holes)

2.2 Address translation with a page table

The logical address is split into a page number **p = a / PAGE_SIZE** and an offset **d = a % PAGE_SIZE**. The page table gives the frame f for page p; the physical address is **f × PAGE_SIZE + d**. The offset is never changed because pages and frames have the same size.

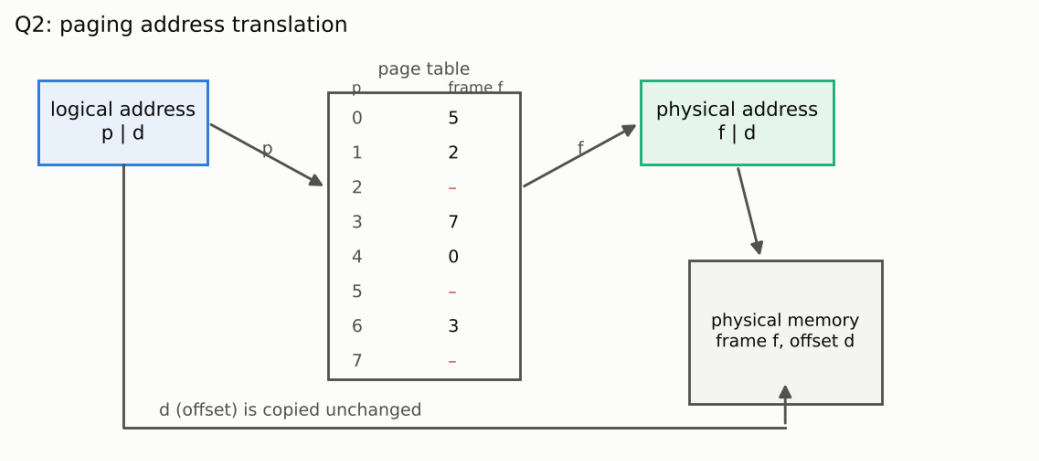


Figure 2.2 – Paging address translation

Complete the Code – TODO 1

paging.c (TODO 1 replaced by working code)

```
#include <stdio.h>
#define PAGE_SIZE 1024
#define PAGES 8
/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};
```

```
int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        unsigned p = a / PAGE_SIZE, d = a % PAGE_SIZE;
        printf("%u -> page %u offset %u : ", a, p, d);
        if (p >= PAGES)              printf("invalid address\n");
        else if (page_table[p] == -1) printf("page fault\n");
        else                         printf("physical %u\n", page_table[p] * PAGE_SIZE + d);
    }
    return 0;
}
```

Compile and run

```
gcc -std=c11 -Wall -Wextra -o paging paging.c && ./paging
```

Output (PAGE_SIZE 1024) – identical to the expected output

```
0 -> page 0 offset 0 : physical 5120
1500 -> page 1 offset 476 : physical 2524
3100 -> page 3 offset 28 : physical 7196
6200 -> page 6 offset 56 : physical 3128
9000 -> page 8 offset 808 : invalid address
2049 -> page 2 offset 1 : page fault
```

PAGE_SIZE = 2048 – hand calculation before running

With 2048-byte pages the 8-entry table covers 16,384 bytes, so address 9000 is now valid. Hand calculation: page = a div 2048, offset = a mod 2048, physical = frame × 2048 + offset.

Logical a	Page p	Offset d	Frame (table)	Physical (by hand)	Program output
0	0	0	5	5×2048+0 = 10240	10240 ✓
1500	0	1500	5	10240+1500 = 11740	11740 ✓
3100	1	3100−2048 = 1052	2	2×2048+1052 = 5148	5148 ✓
6200	3	6200−6144 = 56	7	7×2048+56 = 14392	14392 ✓
9000	4	9000−8192 = 808	0	0×2048+808 = 808	808 ✓
2049	1	1	2	2×2048+1 = 4097	4097 ✓

Output (PAGE_SIZE 2048) – matches the hand calculation

```
0 -> page 0 offset 0 : physical 10240
1500 -> page 0 offset 1500 : physical 11740
3100 -> page 1 offset 1052 : physical 5148
6200 -> page 3 offset 56 : physical 14392
9000 -> page 4 offset 808 : physical 808
2049 -> page 1 offset 1 : physical 4097
```

Observation: with larger pages, addresses 9000 (invalid) and 2049 (page fault) become valid and resident, because the same 8 table entries now cover twice the address range and the page that holds them is bigger.

2.3 Page-table size and effective access time

Page-table size (4 KB pages)

Address space	Number of pages	Entries × entry size	Flat table size per process
32-bit	$2^{32} / 2^{12} = 2^{20} =$ 1,048,576	$2^{20} \times 4$ B	4 MB
48-bit	$2^{48} / 2^{12} = 2^{36} =$ 68,719,476,736	$2^{36} \times 8$ B	512 GB – impossible → multi-level tables

Effective memory-access time (EAT)

Assumptions: memory access 100 ns, TLB lookup 10 ns. **Without a TLB** each access needs one memory access for the page table and one for the data: EAT = 2 × 100 = **200 ns** (single-level), or 5 × 100 = 500 ns with a 4-level table. **With a TLB**: EAT = h(10 + 100) + (1 − h)(10 + 2×100).

Output

```
32-bit, 4KB pages: entries = 2^20 = 1,048,576; size @4B = 4 MB, @8B = 0.01 GB
48-bit, 4KB pages: entries = 2^36 = 68,719,476,736; size @4B = 262,144 MB, @8B = 512.00 GB

EAT without TLB (single-level) = 2*mem = 200 ns   (4-level paging: 5*mem = 500 ns)
EAT with TLB (TLB lookup 10 ns, memory 100 ns, single-level table):
  hit 80%: EAT = 130.0 ns  (slowdown 30.0%);  with 4-level walk on miss: 190.0 ns (90.0%)
  hit 95%: EAT = 115.0 ns  (slowdown 15.0%);  with 4-level walk on miss: 130.0 ns (30.0%)
  hit 99%: EAT = 111.0 ns  (slowdown 11.0%);  with 4-level walk on miss: 114.0 ns (14.0%)
```

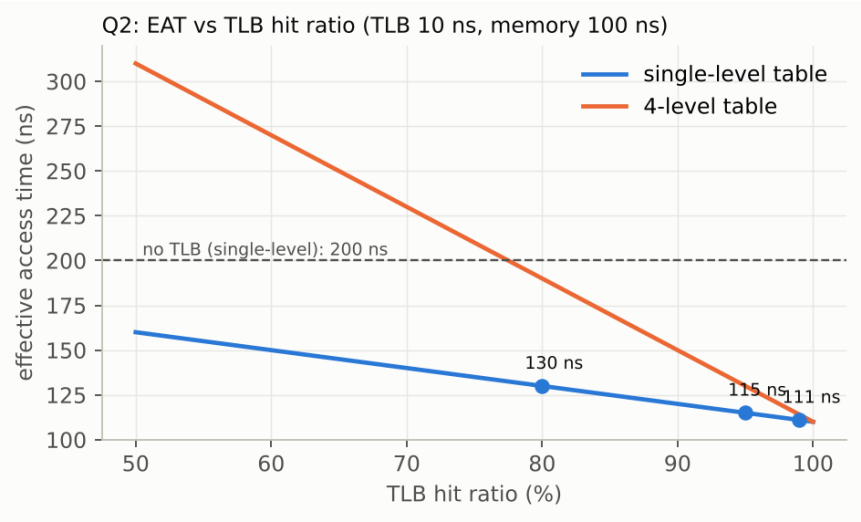


Figure 2.3 – Effective access time against TLB hit ratio

TLB hit ratio	EAT (1 level)	Slow-down	EAT (4 levels)	Slow-down
80 %	130 ns	30 %	190 ns	90 %
95 %	115 ns	15 %	130 ns	30 %
99 %	111 ns	11 %	114 ns	14 %
No TLB	200 ns	100 %	500 ns	400 %

2.4 Evaluation

How paging removes external fragmentation, and why internal fragmentation remains

Memory is divided into equal frames and every process into equal pages, so **any free frame can hold any page** — a process no longer needs contiguous memory, so no unusable holes appear between allocations. But a process's size is rarely an exact multiple of the page size, so its **last page is only partly used**; the unused part is **internal fragmentation** (on average half a page per process).

Page sizes: 1 KB, 4 KB and 2 MB

Page size	Internal fragmentation	Page-table size	TLB reach (64 entries)	Disk I/O per fault	Best for
1 KB	lowest (avg 0.5 KB)	largest (4 M entries per 4 GB)	64 KB – poor	small transfers, high seek overhead per byte	tiny embedded systems
4 KB	low (avg 2 KB)	moderate (1 M entries per 4 GB)	256 KB	good balance	general purpose – the Linux/x86 default

Page size	Internal fragmentation	Page-table size	TLB reach (64 entries)	Disk I/O per fault	Best for
2 MB	high (avg 1 MB)	smallest (2 K entries per 4 GB)	128 MB – excellent	large, slow to read	databases, big arrays, scientific/ML (huge pages)

Hierarchical, hashed and inverted page tables

- **Hierarchical (multi-level)** – the page table is itself paged. x86-64 uses four levels of 9 bits (PML4 → PDPT → PD → PT) + 12-bit offset. Only the parts of the table that are in use are allocated, so a sparse 48-bit space needs kilobytes instead of 512 GB. Cost: up to 4 extra memory accesses on a TLB miss.
- **Hashed** – the virtual page number is hashed into a table of chains; each element holds (VPN, frame, next). Good for sparse address spaces larger than 32 bits; look-up is O(1) on average, O(chain length) worst case.
- **Inverted** – one entry per physical frame, (process id, page number). Table size is proportional to physical memory, not to address space; look-up needs a hash because the table is searched by virtual page. Difficult to share pages (one virtual page per frame). Used by PowerPC and UltraSPARC.

Protection and shared pages

- Each page-table entry has **valid/invalid**, **read/write/execute** and **user/supervisor** bits. The MMU checks them on every access; a violation raises a trap (segmentation fault). The valid bit also marks pages not in memory (page fault).
- **Shared pages**: the page tables of several processes simply point to the same frame. Re-entrant (read-only) code such as libc or an editor is kept once in memory (Q1 showed the same PFN in two processes); writable data is shared only if requested (shared memory) or copy-on-write after `fork()` (Q4).

2.5 Experimental investigation

(a) Internal fragmentation of 100 random processes

100 processes with random sizes between 1 KB and 50 MB (seeded random numbers, total 2,601.5 MB):

frag.c

```
/* Internal fragmentation of 100 processes with random sizes, for 1KB, 4KB, 2MB pages */
#include <stdio.h>
#include <stdlib.h>
int main(void){
  srand(42); enum{N=100}; unsigned long sz[N]; unsigned long tot=0;
```

```
  for(int i=0;i<N;i++){ sz[i]=1024UL+ (unsigned long)rand()%(50UL*1024*1024);
tot+=sz[i]; }  /* 1KB..50MB */
  unsigned long ps[]={1024,4096,2UL*1024*1024};
  const char*nm[]={"1 KB","4 KB","2 MB"};
  printf("100 processes, sizes 1KB..50MB, total = %.2f MB\n",tot/1048576.0);
  printf("%-6s %12s %16s %12s %14s\n","page","pages used","waste (bytes)","avg
waste/proc","waste %");
  for(int k=0;k<3;k++){ unsigned long waste=0,pages=0;
    for(int i=0;i<N;i++){ unsigned long p=(sz[i]+ps[k]-1)/ps[k]; pages+=p;
waste+=p*ps[k]-sz[i]; }
    printf("%-6s %12lu %16lu %12.0f %13.4f%%   page table (8B/entry) = %.2f
MB\n",nm[k],pages,waste,(double)waste/N,100.0*waste/tot,pages*8/1048576.0);}
}
```

Output

```
100 processes, sizes 1KB..50MB, total = 2601.52 MB
page     pages used    waste (bytes) avg waste/proc       waste %
1 KB       2664010         54765         548       0.0020%   page table (8B/entry) =
20.32 MB
4 KB        666039        204269        2043       0.0075%   page table (8B/entry) =
5.08 MB
2 MB          1350     103263725     1032637       3.7855%   page table (8B/entry) =
0.01 MB
```



Figure 2.4 – Internal fragmentation and page-table size for each page size

Page size	Total wasted	Average per process	Waste (% of memory)	Page-table size (8 B/entry)
1 KB	54,765 B	548 B	0.0020 %	20.32 MB
4 KB	204,269 B	2,043 B	0.0075 %	5.08 MB
2 MB	103,263,725 B (≈ 98.5 MB)	1,032,637 B (≈ 1 MB)	3.79 %	0.01 MB

As predicted, the average waste is about half a page per process (548 B ≈ ½ KB, 2,043 B ≈ ½ × 4 KB, 1.03 MB ≈ ½ × 2 MB), while the page-table size falls as the page gets larger. 4 KB is a good trade-off; 2 MB pages waste 98.5 MB here but need 2,000 times fewer table entries than 1 KB pages.

(b) Effective access time for hit ratios 80 %, 95 % and 99 %

See the table and Figure 2.3 in Section 2.3: **130 ns, 115 ns and 111 ns** respectively. Raising the hit ratio from 80 % to 99 % cuts the overhead from 30 % to 11 %, which is why TLBs (and larger TLB reach through bigger pages) matter.

(c) Page size of this Linux system and page faults of a program

getconf PAGESIZE

```
4096
```

Page faults of a program that allocates and touches 64 MB

```
64 MB program touching every page: minor faults = 16445 major faults = 0
expected ~ 64MB/4KB =  16384 (+ start-up faults)
```

`getconf PAGESIZE` gives **4096**. A program touching 64 MB incurred 16,445 minor page faults ≈ 64 MB / 4 KB = 16,384 (plus about 60 start-up faults) and **no major faults**: every first touch of an anonymous page is a (minor) demand-zero fault. The counts come from `getrusage()`, the same counters printed by `/usr/bin/time -v` (that tool was not installed in my environment; on your own machine run `/usr/bin/time -v ./pf` and read 'Minor (reclaiming a frame) page faults').

(d) Start of page, end of page and beyond the last page

edge.c

```
/* Test addresses at start/end of a page and beyond the last page (page size 1024, 8
pages) */
#include <stdio.h>
#define PS 1024
#define N 8
int pt[N]={5,2,-1,7,0,-1,3,-1};
int main(void){
  unsigned t[]={0,1023,1024,2047,2048,7167,7168,8191,8192,100000};
  for(int i=0;i<10;i++){unsigned p=t[i]/PS,d=t[i]%PS;
    printf("%6u: page %u off %4u -> ",t[i],p,d);
    if(p>=N)puts("INVALID (beyond last page, trap)");
    else if(pt[p]<0)puts("PAGE FAULT (not resident)");
    else printf("phys %u\n",pt[p]*PS+d);}
}
```

Output (page size 1024, 8 pages)

```
     0: page 0 off    0 -> phys 5120
```

```
  1023: page 0 off 1023 -> phys 6143
  1024: page 1 off    0 -> phys 2048
  2047: page 1 off 1023 -> phys 3071
  2048: page 2 off    0 -> PAGE FAULT (not resident)
  7167: page 6 off 1023 -> phys 4095
  7168: page 7 off    0 -> PAGE FAULT (not resident)
  8191: page 7 off 1023 -> PAGE FAULT (not resident)
  8192: page 8 off    0 -> INVALID (beyond last page, trap)
100000: page 97 off  672 -> INVALID (beyond last page, trap)
```

The last byte of a page (offset 1023) and the first byte of the next page (offset 0) land in different frames (e.g. 1023 → frame 5, 1024 → frame 2). Page 2 and page 7 are not resident, so addresses in them cause page faults, while any address ≥ 8192 is beyond the table and is rejected as invalid.

2.6 Conclusion – page size for the college server

I would choose **4 KB pages** for the general programs, with an option for 2 MB huge pages (transparent huge pages) for the few very large, long-running programs. Student programs are small and numerous: 4 KB gives tiny internal fragmentation (≈ 2 KB per process), keeps the page-table overhead moderate (5 MB for 2.6 GB of programs) and loads quickly from disk, and it is the size that the hardware, the TLB design and the kernel are optimised for. 1 KB pages would quadruple the page-table memory and TLB pressure for no real gain; 2 MB pages would waste on average 1 MB for each of hundreds of small programs.

2.7 Alternative concept – inverted page table instead of per-process multi-level tables

An inverted page table has one entry (pid, virtual page number, chain pointer) for every physical frame. Translation hashes (pid, page) to a bucket and follows the chain until the entry matching the page is found; the index of the entry is the frame number. I simulated a 1 GB machine (262,144 frames) with random mappings and compared translation costs:

Output of ipt.c

```
physical memory: 262144 frames (1 GB)
inverted table size : 4.0 MB (16 B/entry x frames) + hash anchor 1.0 MB
hash lookup  : 43.1 ns per translation
4-level walk : 85.3 ns per translation (sparse table built for same mappings)
(checksum 524094594566)
```

Criterion	Per-process multi-level (4 levels)	Inverted (hash)
Space	O(number of mapped pages of every process) – grows with the number and size of processes (≥ 4 KB per active process even if tiny)	O(physical frames) = 16 B × 262,144 = 4 MB (+1 MB hash anchors) for 1 GB, independent of processes and address-space size
Translation time	walk of 4 levels, 4 dependent memory accesses: 85.3 ns in the simulation	hash + short chain, about 1–2 accesses: 43.1 ns in the simulation
Big sparse 64-bit address spaces	fine (only used parts exist)	excellent (size independent of address space)
Shared pages	easy – many tables point to one frame	hard – one entry per frame means only one (pid, page) can be recorded
Swapped-out pages	entries can hold the disk location	need an additional external table per process
Hardware support	walkers + page-walk caches in x86/ARM	used by PowerPC / UltraSPARC; must be searched

Big-O: space $O(F)$ vs $O(\Sigma p_i)$ where F = number of frames and p_i = pages used by process i; time $O(1)$ expected for the hash ($O(\text{chain})$ worst case) vs $O(L) = 4$ levels, i.e. constant but with a larger constant. The simulation measures software pointer-chasing, not real MMU hardware (which caches upper levels), so the 2× time advantage should be read as a trend. Because the college server shares compiler and library pages between hundreds of processes, the shared-page problem makes the multi-level table (as used by Linux) the practical choice.

Q3 – Segmentation (6 marks)

3.1 Implementation

(a) (segment, offset) translation with base and limit

A logical address is a pair (s, d). The segment table entry s holds the **base** (start in physical memory) and the **limit** (length). If d ≥ limit the hardware traps; otherwise physical = base + d. A permission field (r/w/x) is checked as well.

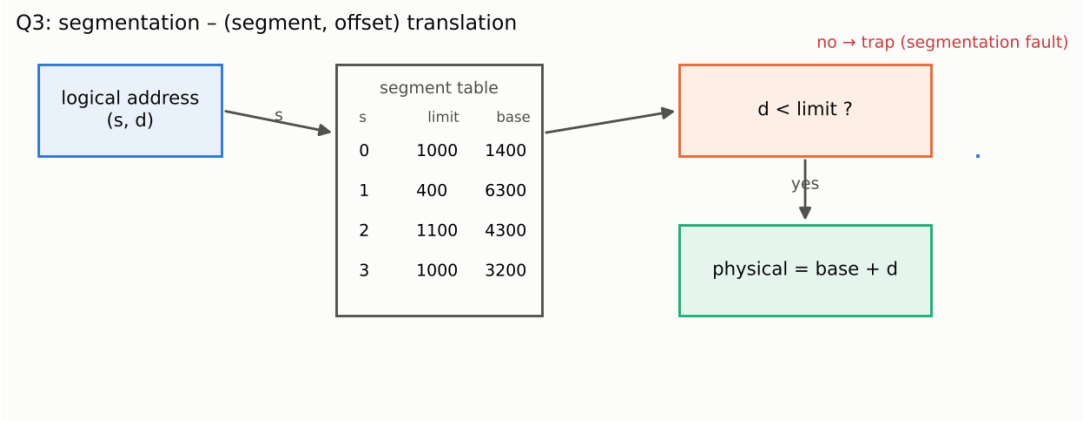


Figure 3.1 – Segment-table translation

Segment table used (addresses in bytes):

Seg	Name	Base	Limit	Permissions
0	code	1400	1000	r-x
1	data	6300	400	rw-
2	heap	4300	1100	rw-
3	stack	3200	1000	rw-
4	shared-lib	5400	500	r-x

seg.c

```
#include <stdio.h>
struct seg { const char*name; unsigned base, limit; const char*perm; };
struct seg st[] = {{"code",1400,1000,"r-x"},{"data",6300,400,"rw-
"},{"heap",4300,1100,"rw-"},{"stack",3200,1000,"rw-"},{"shared-lib",5400,500,"r-
x"}};
int main(void){
  struct { int s; unsigned off; char acc; } t[] = {

{0,53,'r'},{1,399,'w'},{2,1099,'r'},{3,0,'w'},{4,500,'r'},{0,1000,'r'},{1,100,'x'},{3,1500,
'r'}};
  printf("%-3s %-10s %-6s %-5s -> result\n","#","seg","offset","acc");
  for(int i=0;i<8;i++){
    struct seg*s=&st[t[i].s];
    printf("%-3d %-10s %-6u %-5c -> ",i+1,s->name,t[i].off,t[i].acc);
```

```
    if(t[i].off>=s->limit) printf("TRAP: offset %u >= limit %u (segmentation
fault)\n",t[i].off,s->limit);
    else { const char*p=s->perm; int ok =
(t[i].acc=='r'&&p[0]=='r')||(t[i].acc=='w'&&p[1]=='w')||(t[i].acc=='x'&&p[2]=='x');
      if(!ok) printf("TRAP: protection violation (%c on %s segment)\n",t[i].acc,p);
      else printf("physical %u\n",s->base+t[i].off); }
  }
}
```

(b) Translating 8 addresses by hand and verifying with the program

#	(segment, offset), access	Check	Hand result
1	(code, 53), read	53 < 1000	1400 + 53 = 1453
2	(data, 399), write	399 < 400, rw ok	6300 + 399 = 6699
3	(heap, 1099), read	1099 < 1100	4300 + 1099 = 5399
4	(stack, 0), write	0 < 1000	3200 + 0 = 3200
5	(shared-lib, 500), read	500 ≥ 500 (valid offsets are 0–499)	trap – segmentation fault
6	(code, 1000), read	1000 ≥ 1000	trap
7	(data, 100), execute	in limit, but data is rw- (no x)	trap – protection violation
8	(stack, 1500), read	1500 ≥ 1000	trap

Program output – identical to the hand results

```
#  seg        offset acc   -> result
1  code        53     r    -> physical 1453
2  data        399    w    -> physical 6699
3  heap        1099   r    -> physical 5399
4  stack       0      w    -> physical 3200
5  shared-lib 500    r     -> TRAP: offset 500 >= limit 500 (segmentation fault)
6  code        1000   r    -> TRAP: offset 1000 >= limit 1000 (segmentation fault)
7  data        100    x    -> TRAP: protection violation (x on rw- segment)
8  stack       1500   r    -> TRAP: offset 1500 >= limit 1000 (segmentation fault)
```

(c) What happens when the offset exceeds the limit

The MMU raises an addressing-error trap. The OS trap handler finds that the address does not belong to any valid segment and terminates the process (rows 5, 6 and 8 above). No physical memory outside the segment is touched, so one process cannot read or damage another process's memory.

(d) Segmentation with paging (Intel x86)

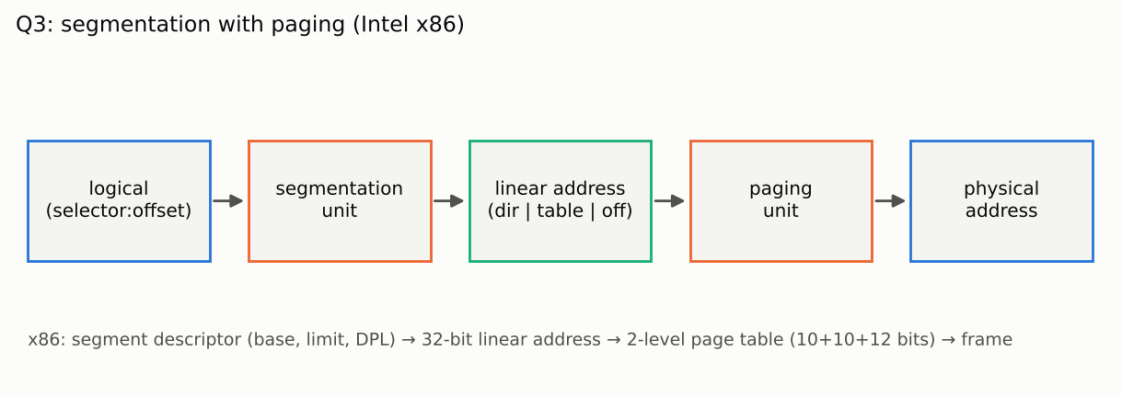


Figure 3.2 – x86: a segment selects a descriptor, the sum is a linear address, paging translates it

The CPU takes the segment selector from a segment register (CS, DS, SS…), looks up the **descriptor** in the GDT/LDT (base, limit, privilege level) and checks the offset against the limit. Base + offset gives a **32-bit linear address**, which is split into 10 bits directory, 10 bits table and 12 bits offset and translated by the two-level page table into a physical address. Each segment is therefore itself paged, so segments need not be contiguous in physical memory and external fragmentation disappears. (x86-64 keeps segmentation only in a 'flat' form: bases are 0 for code/data/stack and paging does the real work.)

(e) Segments of a running Linux program (/proc/<pid>/maps)

layout.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int g_init=5; int g_bss; const char ro[]="read only";
int main(void){ int local; void*h=malloc(1<<20);
  printf("code(main)=%p  rodata=%p  data=%p  bss=%p  heap=%p
stack=%p\n",(void*)main,(void*)ro,(void*)&g_init,(void*)&g_bss,h,(void*)&local);
  FILE*f=fopen("/proc/self/maps","r"); char l[400]; while(fgets(l,sizeof l,f))
if(!strstr(l,"libc")&&!strstr(l,"ld-linux")) fputs(l,stdout); return 0; }
```

Output (non-PIE build so low addresses are easy to read)

```
code(main)=0x4011f6  rodata=0x402008  data=0x404048  bss=0x40405c
heap=0xc0b02a0  stack=0x7ffc4aba55ec
00400000-00401000 r--p 00000000 fe:00 795195
/home/claude/os/Q3/layout
00401000-00402000 r-xp 00001000 fe:00 795195
/home/claude/os/Q3/layout
00402000-00403000 r--p 00002000 fe:00 795195
/home/claude/os/Q3/layout
00403000-00404000 r--p 00002000 fe:00 795195
/home/claude/os/Q3/layout
```

```
00404000-00405000 rw-p 00003000 fe:00 795195
/home/claude/os/Q3/layout
0c0b0000-0c1d1000 rw-p 00000000 00:00 0                  [heap]
7f6335a06000-7f6335a13000 rw-p 00000000 00:00 0
7f6335a97000-7f6335a9a000 rw-p 00000000 00:00 0
7f6335aa8000-7f6335aaa000 rw-p 00000000 00:00 0
7f6335aaa000-7f6335aae000 r--p 00000000 00:00 0          [vvar]
7f6335aae000-7f6335ab0000 r--p 00000000 00:00 0          [vvar_vclock]
7f6335ab0000-7f6335ab2000 r-xp 00000000 00:00 0          [vdso]
7ffc4ab86000-7ffc4abac000 rw-p 00000000 00:00 0          [stack]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0  [vsyscall]
```

Address range	Permissions	Segment
0x400000–0x401000 / 0x401000–0x402000	r--p / r-xp	ELF header and code (text) – read-only, executable, shared
0x402000–0x403000	r--p	read-only data (`const char ro[]`)
0x404000–0x405000	rw-p	initialised data (`g_init`) and bss (`g_bss`)
0xc0b0000–0xc1d1000	rw-p	[heap] – `malloc`
0x7ffc4ab86000–0x7ffc4abac000	rw-p	[stack] – grows downwards
0x7f63…	r-xp/rw-p	shared libraries (libc, ld.so, filtered out above), [vdso]

3.2 Evaluation

Segmentation matches the programmer's view of memory

A programmer thinks of a program as a collection of named units — main program, functions, stack, heap, global variables, arrays — each of different length and with different rules. Segmentation gives each unit its own address space (s, d): code is read-only and executable, the stack can grow, the data is non-executable. Paging alone sees only one flat, uniform array of pages and does not know which part is code and which is data.

Protection and sharing

- Every segment-table entry has its own **limit** (bounds check) and **access rights** (r/w/x), so protection follows the logical structure: code segments are read-only, data is non-executable (W^X).
- **Sharing** is done at the level of whole segments: two processes put the same base/limit in their tables (e.g. the compiler's code segment or the shared library).

Because the shared object is one logical unit, the same segment number and offsets are meaningful for both.

External fragmentation of segmentation

Segments have varying lengths and need contiguous physical memory, so loading and removing them leaves holes — **external fragmentation** (see 3.3). Compaction can recover the space but copies memory and is slow; paging the segments (segmented paging) removes the problem.

Comparison: pure paging, pure segmentation and segmented paging

Property	Pure paging	Pure segmentation	Segmented paging
Unit	fixed-size pages	variable-size segments	segments divided into pages
Programmer's view	invisible	visible (code/data/stack)	visible
External fragmentation	none	yes	none
Internal fragmentation	yes (last page)	none	yes (last page of each segment)
Protection / sharing	per page (bits in PTE)	per segment – natural	per segment, enforced through pages
Table size	large if flat (4 MB/process for 32-bit) – multi-level	tiny (one entry per segment)	segment table + one page table per segment
Translation	1 table walk (or several levels)	1 table look-up + compare	2 steps (segment, then page) – slowest
Used by	Linux/x86-64, ARM, almost all modern OSs	early systems (Burroughs, early PDP)	x86-32 (Multics, OS/2 style)

3.3 Experimental investigation

(a) Triggering a segmentation fault and what the OS did

segv.c

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>
```

```
static void h(int s, siginfo_t*i, void*c){ (void)c; char b[128]; int n=snprintf(b,sizeof
b,"caught signal %d (SIGSEGV), faulting address %p, si_code=%s\n",s,i->si_addr,i-
>si_code==SEGV_MAPERR?"SEGV_MAPERR (no mapping)":"SEGV_ACCERR
(protection)"); write(1,b,n); _exit(139);}
int main(int argc,char**argv){ (void)argv;
  struct sigaction sa; memset(&sa,0,sizeof sa); sa.sa_sigaction=h;
sa.sa_flags=SA_SIGINFO; sigaction(SIGSEGV,&sa,0);
  if(argc>1){ const char*lit="hello"; printf("writing into read-only string literal at
%p...\n",(const void*)lit); fflush(stdout); *(char*)lit='H'; }
  else { int*p=(int*)0x10; printf("dereferencing unmapped address %p...\n",(void*)p);
fflush(stdout); *p=1; }
}
```

Output

```
dereferencing unmapped address 0x10...
caught signal 11 (SIGSEGV), faulting address 0x10, si_code=SEGV_MAPERR (no
mapping)
writing into read-only string literal at 0x55e13d7ef07d...
caught signal 11 (SIGSEGV), faulting address 0x55e13d7ef07d,
si_code=SEGV_ACCERR (protection)
shell exit status = 139 (128+11 = SIGSEGV)
```

Two faults were provoked: writing to address `0x10` (no mapping – `SEGV_MAPERR`) and writing into a read-only string literal (page mapped but not writable – `SEGV_ACCERR`). In both cases the sequence is: (1) the CPU/MMU finds no valid translation or a permission violation and raises a **page-fault exception**; (2) the kernel's fault handler looks up the faulting address in the process's memory areas (the lines of `/proc/<pid>/maps`); (3) the address is not in any area (or the access type is not allowed), so it is not a legal demand-paging fault; (4) the kernel sends **SIGSEGV** to the process; (5) the default action terminates the process (exit status 139 = 128 + 11, as shown for the plain program) and may write a core dump. My program caught the signal with a handler to print the faulting address and the reason code.

(b) Fragmentation after loading and removing 20 segments

20 segments of random size (50–400 KB) were loaded into 4 MB by first-fit; 10 randomly chosen segments were then removed:

frag.py

```
import random
random.seed(7)
MEM=4096   # KB
holes=[(0,MEM)]; segs={}
def alloc(i,sz):
    global holes
    for k,(s,l) in enumerate(holes):        # first fit
```

```
        if l>=sz:
            segs[i]=(s,sz); holes[k]=(s+sz,l-sz)
            if holes[k][1]==0: holes.pop(k)
            return True
    return False
def free(i):
    global holes
    s,l=segs.pop(i); holes.append((s,l)); holes.sort()
    m=[]
    for h in holes:
        if m and m[-1][0]+m[-1][1]==h[0]: m[-1]=(m[-1][0],m[-1][1]+h[1])
        else: m.append(h)
    holes=m
sizes={i:random.randint(50,400) for i in range(20)}
print("Loaded 20 segments (KB):",list(sizes.values()))
for i,s in sizes.items(): alloc(i,s) or print("fail",i)
rem=random.sample(range(20),10)
print("Removing segments:",sorted(rem))
for i in rem: free(i)
fr=sum(l for s,l in holes); big=max(l for s,l in holes)
print("Free holes (start,size KB):",holes)
print(f"Total free = {fr} KB in {len(holes)} holes; largest hole = {big} KB")
print(f"External fragmentation = 1 - largest/total_free = {(1-big/fr)*100:.1f}%")
... (8 more lines)
```

Output

```
Loaded 20 segments (KB): [215, 127, 252, 383, 74, 87, 324, 98, 237, 348, 79, 309, 159, 69, 94, 272, 264, 85, 173, 96]
Removing segments: [0, 1, 3, 7, 9, 10, 12, 13, 14, 17]
Free holes (start,size KB): [(0, 342), (594, 383), (1462, 98), (1797, 427), (2533, 322), (3391, 85), (3745, 351)]
Total free = 2008 KB in 7 holes; largest hole = 427 KB
External fragmentation = 1 - largest/total_free = 78.7%
A new segment of 428 KB (<= total free 2008 KB) cannot be loaded without compaction -> FAILS
Paging (4 KB) on the same 20 segments: external frag = 0, internal waste = 27 KB total (avg 1.4 KB per segment)
```

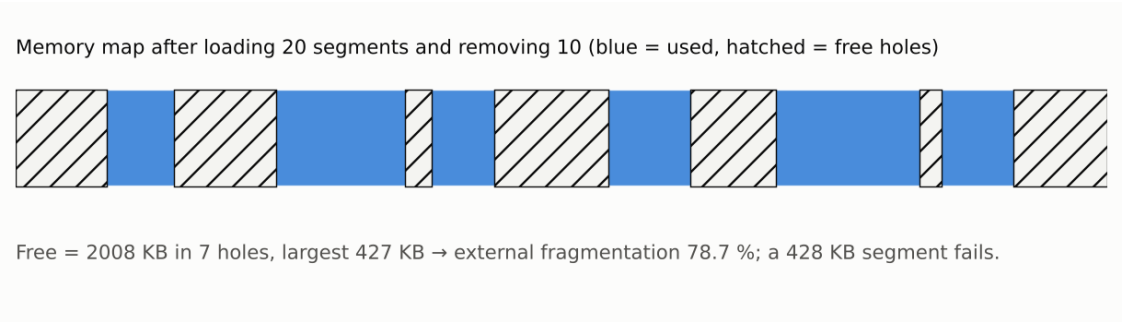


Figure 3.3 – Memory map after removing 10 of 20 segments

Total free memory is 2,008 KB, but it is spread over **7 holes with the largest only 427 KB**: external fragmentation = 1 − 427/2008 = **78.7 %**. A new 428 KB segment fails although 2,008 KB are free. Paging the same 20 segments with 4 KB pages would waste only 27 KB in total (1.4 KB per segment) and never fails while free frames remain.

(c) Size of the tables needed in each scheme

Example process (32-bit): code 200 KB, data 100 KB, heap 1 MB, stack 64 KB.

Output

```
Process:  {'code': 200, 'data': 100, 'heap': 1024, 'stack': 64} KB
Pure segmentation : 4 entries x (base 4B + limit 4B) =  32 bytes
Pure paging (flat, 32-bit, 4KB): 2^20 entries x 4B = 4 MB regardless of process size
Pure paging (two-level, only used pages):  347 PTEs x 4B +  top dir 4 KB + a few 4KB leaf tables ~ 20480 bytes
Segmented paging: 4 segment entries + one small page table per segment: 1420 bytes
```

Scheme	Table size for this process
Pure segmentation	4 entries × 8 B = 32 B
Pure paging – flat	2^{20} × 4 B = 4 MB (independent of process size)
Pure paging – two-level	≈ 20 KB (one 4 KB directory + a few 4 KB second-level tables)
Segmented paging	32 B + 347 PTEs × 4 B ≈ 1.4 KB

3.4 Conclusion – why do modern operating systems rely mainly on paging?

Paging has **no external fragmentation**, allocates memory in fixed units that are simple and fast to manage (a free-frame list), and makes **demand paging and swapping** trivial because every unit has the same size. Segmentation's benefits (protection and sharing by logical unit) can be reproduced with per-page permission bits and by grouping pages into regions (Linux's `vm_area_struct`), while its variable-size allocation causes the 78.7 % external fragmentation measured above. Also, 64-bit x86 dropped segment bases and limits, so the compiler/OS portability is simpler with one mechanism. Linux therefore uses a flat paged model with permissions on pages.

3.5 Alternative concept – pure paging instead of segmentation

Criterion	Segmentation (measured)	Pure paging (4 KB)
Fragmentation	external 78.7 % (2,008 KB free, largest hole 427 KB)	external 0; internal 27 KB for the 20 segments (average 1.4 KB, ≤ 4 KB each) – O(P) worst case, 0.5 page per object on average

Criterion	Segmentation (measured)	Pure paging (4 KB)
Table size	32 B per process (4 entries)	flat 4 MB (32-bit) or ≈ 20 KB multi-level; O(N) for N mapped pages vs O(S) segments
Translation time	1 table look-up + limit compare, O(1)	1 – 4 memory accesses without TLB, O(levels) – hidden by TLB hits (see Q2: 111 ns at 99 % hits)
Code sharing	share a segment (1 table entry)	share pages: every process's page table has entries for each shared page, O(pages) per sharer
Protection	one rights field per segment	rights bits repeated in every PTE of a region – the OS must set many entries, but gives finer granularity

Pure paging is slightly more work to set up for sharing and protection (many PTEs instead of one segment entry) and its tables are larger, but it eliminates external fragmentation completely, and TLBs make the time per translation almost equal. That is the trade that modern systems have chosen.

Q4 – Virtual Memory and Demand Paging (8 marks)

4.1 Implementation

(a) Steps of handling a page fault

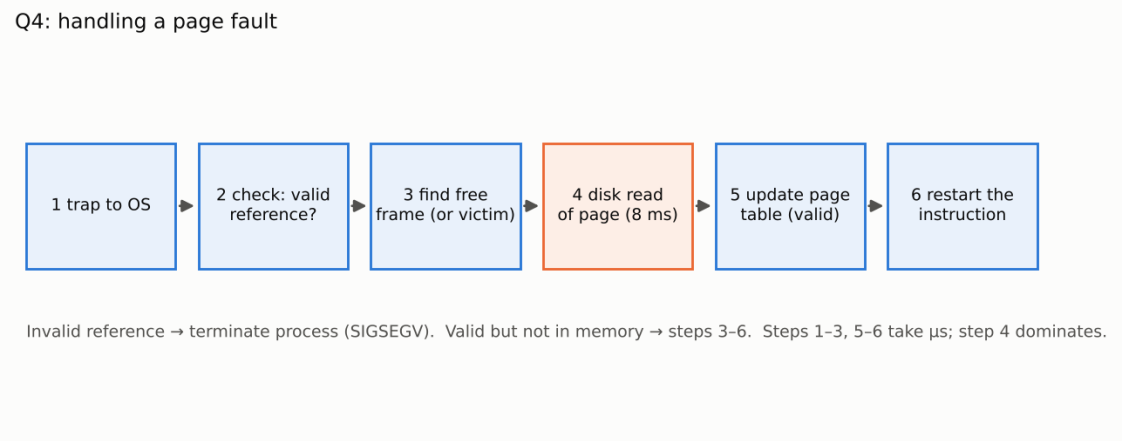


Figure 4.1 – Page-fault handling

1. The process references a page whose valid bit is 0; the MMU raises a **trap** to the OS.
2. The OS checks the process's page table / memory map: if the reference is **illegal** the process is terminated (SIGSEGV); if it is legal but the page is not in memory, continue.
3. The OS finds a **free frame** (if none is free, a page-replacement algorithm picks a victim, and a dirty victim is written to disk first).
4. The OS schedules a **disk read** of the page into the frame (the process waits; the CPU runs another process).
5. When the I/O completes the OS **updates the page table** (frame number, valid bit = 1) and the frame table.
6. The faulting instruction is **restarted**; it now finds the page in memory.

(b) Effective access time (memory 100 ns, fault service 8 ms)

EAT = (1 − p) × 100 ns + p × 8,000,000 ns.

Output

```
EAT = (1-p)*100ns + p*8,000,000ns
p = 1 in 1,000     : EAT =     8099.90 ns   slow-down =    81.0x
p = 1 in 100,000   : EAT =      180.00 ns   slow-down =     1.8x
p = 1 in 1,000,000 : EAT =      108.00 ns   slow-down =     1.1x

For <=10% slow-down: EAT<=110 ns -> p <= 10/(8,000,000-100) = 1.250e-06  ~ 1 fault in 799,990 accesses
```

Page-fault rate p	EAT	Slow-down
1 in 1,000	8,099.9 ns	81×
1 in 100,000	180 ns	1.8×
1 in 1,000,000	108 ns	1.08×

(c) Copy-on-write and why fork() is fast

`fork()` must create a copy of the parent's address space. Instead of copying every page, the kernel copies only the **page tables** and marks all writable pages **read-only in both processes**. The first time either process writes, a protection fault occurs; the kernel allocates a new frame, copies that one page, and gives the writer its own copy. Pages that are only read (or never touched) are never copied. This is why `fork()` of a 800 MB process takes about 10 ms instead of the time to copy 800 MB, and why `fork()`+`exec()` wastes almost nothing.

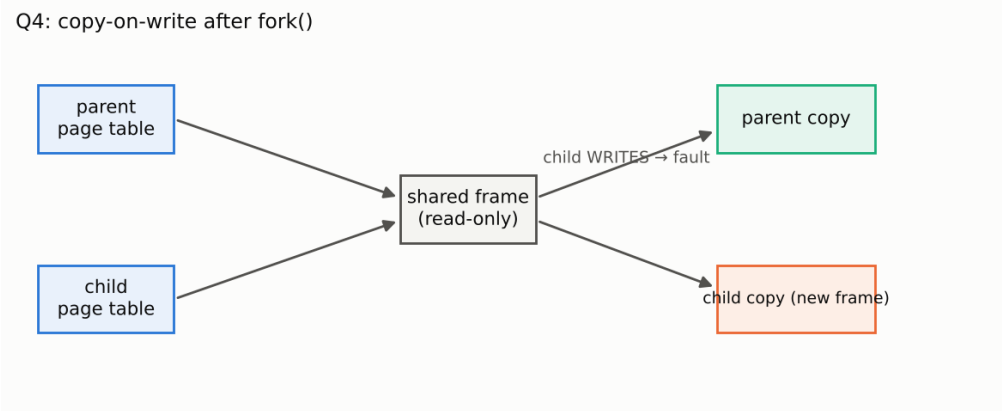


Figure 4.2 – Copy-on-write

(d) Thrashing and the working-set model

Thrashing happens when a process (or the whole system) does not have enough frames for the pages it is actively using, so it faults continuously: CPU utilisation falls because processes are waiting for the disk, and a naïve scheduler then admits even more processes, making things worse. The **working-set model** (Denning) defines $WS(t, \Delta)$ as the set of pages referenced in the last Δ references. If $\Sigma|WS_i| \leq$ number of frames, the system runs well; if $\Sigma|WS_i| >$ frames, the OS must **suspend** a process (swap it out) so the others keep their working sets. This works because programs show **locality**: they use a small group of pages for a while and then move to another group.

4.2 Evaluation

How a 24 GB program can run in 16 GB of RAM

With demand paging the program's 24 GB exist only as a **virtual address space**; each page is loaded when first touched. At any moment a program needs only its current **working set** (the code and data it is using now), not all 24 GB. Pages not currently

used stay on disk (swap or the image file) and unused ones are replaced. Because of locality, faults are rare, so the program runs almost at memory speed — as long as its working set fits in the available frames and the OS is not thrashing.

Maximum acceptable page-fault rate for a 10 % slow-down

EAT ≤ 1.10 × 100 = 110 ns → 100 + p × (8,000,000 − 100) ≤ 110 → **p ≤ 10 / 7,999,900 = 1.25 × 10^{-6}**, i.e. fewer than **about 1 fault per 800,000 memory accesses** (1.25 faults per million).

Frame allocation – equal and proportional

Example: 62 free frames, two processes of 10 pages and 127 pages (137 total).

Method	Rule	P1 (10 pages)	P2 (127 pages)
Equal	m / n frames each	31	31 (too few for P2, wasted on P1 which only needs 10)
Proportional	$a_i = (s_i / S) \times m$	10/137 × 62 ≈ 4	127/137 × 62 ≈ 57

Proportional allocation gives large processes more frames; it can be combined with priority (higher-priority processes get more). Every process also needs a **minimum** number of frames (determined by the instruction set) to run at all.

How the OS detects and stops thrashing

- **Detection:** low CPU utilisation together with a very high paging rate; page-fault frequency (PFF) above an upper threshold; Σ working sets > available frames. Linux exposes memory ‘pressure stall information’ in /proc/pressure/memory.
- **Stopping it:** (1) **local replacement** so one thrashing process cannot steal frames from others; (2) **working-set** or **PFF** control – give a process more frames if its fault rate is too high, take frames away if it is too low, and if no frames are free **suspend** a process; (3) reduce the degree of multiprogramming; (4) in the extreme, the OOM killer terminates a process.

4.3 Experimental investigation

(a) Sequential and random access – page-fault counts

touch.c

```
/* Touch memory sequentially or randomly; count page faults and time.
   usage: ./touch <MB> seq|rand [file]   (file => file-backed mmap, can be evicted &
re-read under a memory limit) */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#include <time.h>
#include <sys/mman.h>
#include <sys/resource.h>
static double now(void){ struct timespec t;
clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec+t.tv_nsec*1e-9; }
int main(int argc,char**argv){
  if(argc<3) return 1;
  size_t n=(size_t)atol(argv[1])<<20; int seq=!strcmp(argv[2],"seq"); long
pg=sysconf(_SC_PAGESIZE); size_t np=n/pg;
  char*p;
  if(argc>3){ int fd=open(argv[3],O_RDWR|O_CREAT,0600); if(ftruncate(fd,n))
return 2; p=mmap(0,n,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0); }
  else
p=mmap(0,n,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-
1,0);
  if(p==MAP_FAILED){perror("mmap");return 1;}
  struct rusage a,b; srand(1);
  /* phase 1: first touch of every page (demand-zero / demand-load) */
  getrusage(RUSAGE_SELF,&a); double t0=now();
  for(size_t i=0;i<np;i++) p[i*pg]++;
  double t1=now(); getrusage(RUSAGE_SELF,&b);
  long f1=b.ru_minflt-a.ru_minflt, m1=b.ru_majflt-a.ru_majflt;
  /* phase 2: np further accesses in chosen order */
  getrusage(RUSAGE_SELF,&a); double t2=now();
  size_t idx=0; for(size_t i=0;i<np;i++){ idx = seq? i : (size_t)(((unsigned
long)rand()<<15 ^ rand())%np); p[idx*pg]++; }
  double t3=now(); getrusage(RUSAGE_SELF,&b);
  printf("%5zu MB %-4s | pass1: minor %7ld major %6ld %7.3fs | pass2: minor %7ld
major %6ld %7.3fs\n",n>>20,argv[2],f1,m1,t1-t0,b.ru_minflt-a.ru_minflt,b.ru_majflt-
a.ru_majflt,t3-t2);
  return 0; }
```

Output – 256 MB of anonymous memory, no memory limit

```
== anonymous memory, no limit, 256 MB ==
  256 MB seq  | pass1: minor   65538 major      0   0.152s | pass2: minor       0 major
0   0.001s
  256 MB rand | pass1: minor   65538 major      0   0.108s | pass2: minor       0 major
0   0.005s
```

In both orders the first pass takes **65,538 minor faults ≈ 256 MB / 4 KB = 65,536** (one demand-zero fault per page, no disk read, 0 major faults). When everything fits in RAM the access order does not change the fault count; the second pass has no faults at all.

(b) Run time as memory grows beyond the available RAM

The VM has no swap device, so anonymous memory cannot be paged out. I therefore mapped a **file** into memory (mmap) and put the process into a **memory cgroup limited to 100 MB**: pages beyond the limit are evicted from memory and have to be re-read from disk on the next touch (major faults) — exactly the behaviour of demand paging.

Memory limit used

```
mkdir /sys/fs/cgroup/memory/osq4; echo 100M >
/sys/fs/cgroup/memory/osq4/memory.limit_in_bytes
bash -c 'echo $$ > /sys/fs/cgroup/memory/osq4/cgroup.procs; exec ./touch 160 rand
big.dat'
```

Output (pass 2 = as many further accesses as pages, in the chosen order)

```
== file-backed mmap under 100 MB memory limit (cgroup) ==
  40 MB rand | pass1: minor    360 major     1   0.009s | pass2: minor      0 major
0   0.001s
  80 MB rand | pass1: minor    680 major     1   0.019s | pass2: minor      0 major
0   0.001s
 160 MB rand | pass1: minor   1320 major     1   0.074s | pass2: minor  23270 major
16235   1.425s
 320 MB rand | pass1: minor   2600 major     1   0.257s | pass2: minor  22932 major
57482   3.713s
  40 MB seq  | pass1: minor    360 major     1   0.011s | pass2: minor      0 major     0
0.000s
  80 MB seq  | pass1: minor    680 major     1   0.024s | pass2: minor      0 major     0
0.000s
 160 MB seq  | pass1: minor   1320 major     1   0.092s | pass2: minor   1318 major
1   0.122s
 320 MB seq  | pass1: minor   2600 major     1   0.201s | pass2: minor   2598 major
1   0.259s
```

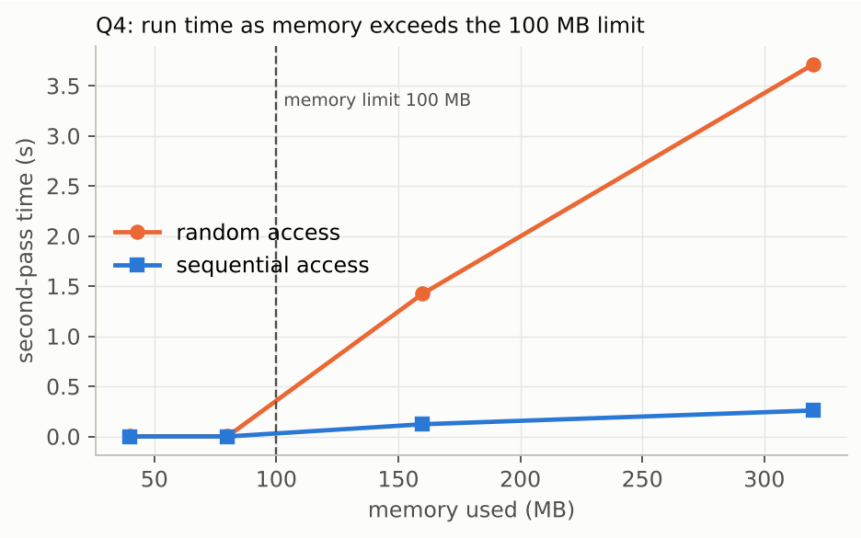


Figure 4.3 – Run time of the second pass against memory used

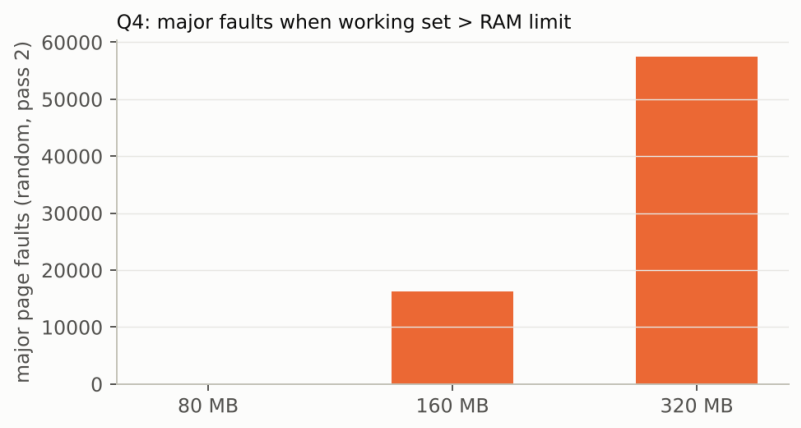


Figure 4.4 – Major page faults of random access

While the data (40, 80 MB) fits in the 100 MB limit, a second pass takes ≈ 1 ms with 0 faults. Once it exceeds the limit the random-access program **thrashes**: 160 MB → 16,235 major faults and 1.4 s; 320 MB → 57,482 major faults (0.70 per access) and **3.7 s — about 45 µs per access instead of ≈ 50 ns, roughly 900× slower per access**. Sequential access is almost unaffected (0.12 s and 0.26 s, only 1 major fault) because the kernel reads ahead and the next page is already coming; this is the effect of locality and of access pattern. The disk here is a fast virtual SSD (≈ 60 µs per fault); with the 8 ms of a hard disk the 57,482 faults alone would take 57,482 × 8 ms ≈ 460 s (nearly 8 minutes).

> Because Linux maps in 16 pages per fault ('fault-around'), the minor-fault counts of file-backed memory are lower than pages/1.

(c) Copy-on-write with fork() – memory before and after writing

cow.c

```
/* Copy-on-write demo: child's own (private) memory before/after writing */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/mman.h>
static double now(void){ struct timespec t;
clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec*1e3+t.tv_nsec*1e-6; }
static void show(const char*tag){
  FILE*f=fopen("/proc/self/smaps_rollup","r"); char l[256]; long
rss=0,pss=0,pd=0,sh=0;
  while(fgets(l,sizeof l,f)){ sscanf(l,"Rss: %ld",&rss); sscanf(l,"Pss: %ld",&pss);
sscanf(l,"Private_Dirty: %ld",&pd); sscanf(l,"Shared_Dirty: %ld",&sh);} fclose(f);
  printf("  [%-22s] pid %d  Rss %7ld kB  Pss %7ld kB  Private_Dirty %7ld kB
Shared_Dirty %7ld kB\n",tag,getpid(),rss/1,pss,pd,sh); fflush(stdout);}
int main(int argc,char**argv){
  size_t mb = argc>1?atol(argv[1]):200, n=mb<<20;
char*p=mmap(0,n,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYM
OUS,-1,0);
  memset(p,1,n); printf("parent allocated and touched %zu MB\n",mb); show("parent
before fork");
  double t=now(); pid_t c=fork();
  if(c==0){ show("child right after fork"); volatile char s=0; for(size_t
i=0;i<n;i+=4096) s+=p[i]; show("child after READING all");
    for(size_t i=0;i<n/2;i+=4096) p[i]=2;
    show("child after writing 50%"); _exit(0); }
  double tf=now()-t; waitpid(c,0,0); printf("fork() call took %.3f ms for a %zu MB
process\n",tf,mb); return 0; }
... (1 more lines)
```

Output

```
parent allocated and touched 200 MB
  [parent before fork    ] pid 198  Rss  206532 kB  Pss  205174 kB  Private_Dirty
204936 kB  Shared_Dirty       0 kB
  [child right after fork] pid 200  Rss  205600 kB  Pss  102583 kB  Private_Dirty     40
kB  Shared_Dirty  204892 kB
  [child after READING all] pid 200  Rss  206036 kB  Pss  102656 kB  Private_Dirty
44 kB  Shared_Dirty  204888 kB
  [child after writing 50%] pid 200  Rss  206036 kB  Pss  153856 kB  Private_Dirty
102444 kB  Shared_Dirty  102488 kB
fork() call took 2.433 ms for a 200 MB process
fork() call took 0.636 ms for a 50 MB process
fork() call took 2.699 ms for a 200 MB process
fork() call took 10.638 ms for a 800 MB process
```

Moment	Child Pss	Child Private_Dirty	Meaning
right after fork	102,583 kB	40 kB	all 200 MB shared with the parent (Pss = half of it)
after reading all pages	102,656 kB	44 kB	reading does not copy
after writing 50 % of pages	153,856 kB	102,444 kB	≈ 100 MB copied: one copy per written page

`fork()` itself took 0.64 ms (50 MB), 2.7 ms (200 MB) and 10.6 ms (800 MB): it grows with the number of page-table entries to copy, not with the data.

(d) Page faults against the number of frames for a reference string

Reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 (20 references), computed with the simulator of Q5:

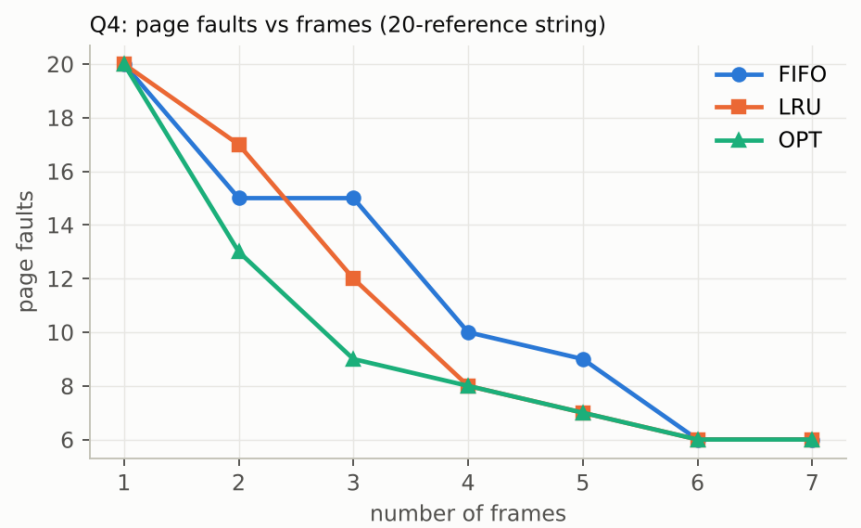


Figure 4.5 – Page faults against frames: FIFO 20, 15, 15, 10, 9, 6, 6; LRU 20, 17, 12, 8, 7, 6, 6; OPT 20, 13, 9, 8, 7, 6, 6 for 1–7 frames

Faults fall as frames increase, and above 6 frames only the 6 compulsory (first-use) faults remain (there are only 6 distinct pages in this string; the curve is 7 0 1 2 3 4 → 6). The non-monotonic behaviour of FIFO (Belady's anomaly) is shown in Q5.

4.4 Conclusion – more RAM or better locality?

Improve locality first, then buy RAM if the working set still exceeds it. My measurements show that the access pattern, not only the size, decides the penalty: with data 3.2× the memory limit, sequential access took 0.26 s while random access took 3.7 s (14× slower) and 57,482 major faults. The radiologist's image-processing code usually works on slices or tiles, so processing the 24 GB data **tile by tile in the order it is stored** (blocking, streaming, prefetching) can reduce the page-fault rate below the

1.25×10^{-6} needed for a 10 % slow-down, at no hardware cost. But if the working set itself (the data needed at once) is larger than 16 GB, no locality improvement will help and more RAM is the only cure. The two are complementary; software changes are cheap and can be tested now, RAM is the fix if tests still show a high major-fault rate (/proc/pressure/memory, vmstat).

4.5 Alternative concept – run on 32 GB of RAM instead of relying on demand paging

With 32 GB the whole 24 GB fits in memory. After the one-time load of the image there are no major faults, $p \approx 0$ and EAT ≈ **100 ns**. With demand paging on the 16 GB laptop, EAT depends on the fault rate that is actually achieved (using the 8 ms service time):

Case	Fault rate p	EAT	Slow-down vs 32 GB
32 GB RAM	≈ 0	100 ns	1×
16 GB, good locality (measured sequential: 1 major fault in 40,960 accesses)	2.4×10^{-5}	$100 + 2.4\times10^{-5} \times 8\times10^{6} \approx 295$ ns	≈ 3×
16 GB, poor locality (measured random 160 MB/100 MB limit: 16,235 faults in 40,960 accesses)	0.40	≈ 3.2 ms	≈ 32,000×
16 GB, tuned program with $p = 10^{-6}$	10^{-6}	108 ns	1.08×

Cost trade-off: extra RAM is a one-time cost, which on a laptop may not be possible at all (soldered memory) or may require a new machine; the slow run costs the radiologist's time on every scan. With T seconds saved per scan, N scans per day and a staff cost of C per hour, the RAM pays back when $N \times T \times C \times$ (working days) exceeds the price quoted for the upgrade. Since the measured penalty for poor locality is huge (hundreds of times per access in my test), buying RAM (or tuning the program) is clearly better than running a 24 GB random-access job in 16 GB; if locality is good the penalty is small and the extra RAM may be unnecessary.
Complexity: demand paging time is $O(n \times EAT)$ with EAT depending on p; with enough RAM it is $O(n \times t_{mem})$; space is O(working set) vs O(whole data set) resident.

Q5 – Page-Replacement Techniques (8 marks)

5.1 Implementation – FIFO, LRU and Optimal

Three algorithms were implemented in replace.c (FIFO circular queue, LRU with a 'last used' time per frame, Optimal looking forward for the page used farthest in the future). The only change to the given code is the completed **TODO 1** in lru().

Complete the Code – TODO 1 (LRU)

The completed TODO

```
        /* TODO 1 (completed) */
        if (used < frames) { f[used] = ref[i]; last[used] = i; used++; }   /* a frame is free:
use it */
        else {                                              /* replace the least recently used */
            int victim = 0;
            for (int j = 1; j < frames; j++) if (last[j] < last[victim]) victim = j;
            f[victim] = ref[i]; last[victim] = i;                    /* record last use of the
frame used */
        }
```

replace.c – complete program

```
#include <stdio.h>
#define MAXF 8
int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}
int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}
int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        /* TODO 1 (completed) */
```

```
        if (used < frames) { f[used] = ref[i]; last[used] = i; used++; }
        else {
            int victim = 0;
            for (int j = 1; j < frames; j++) if (last[j] < last[victim]) victim = j;
            f[victim] = ref[i]; last[victim] = i;
        }
    }
    return faults;
}
int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len; /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
            if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults;
}
int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int len = 20;
    for (int fr = 3; fr <= 4; fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
               lru(ref, len, fr), optimal(ref, len, fr));
    int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
           fifo(b, 12, 3), fifo(b, 12, 4));
    return 0;
}
```

Compile and run

```
gcc -std=c11 -Wall -Wextra -o replace replace.c && ./replace
```

Output – identical to the expected output

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Hand calculation to verify the program (3 frames)

Reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 with 3 frames. After each reference the frame contents are shown (FIFO: oldest first leaves; LRU: the page whose last use is oldest leaves).

#	Ref	FIFO frames	FIFO	LRU frames	LRU
1	7	7	FAULT	7	FAULT
2	0	7 0	FAULT	7 0	FAULT
3	1	7 0 1	FAULT	7 0 1	FAULT
4	2	2 0 1	FAULT	2 0 1	FAULT
5	0	2 0 1	hit	2 0 1	hit
6	3	2 3 1	FAULT	2 0 3	FAULT
7	0	2 3 0	FAULT	2 0 3	hit
8	4	4 3 0	FAULT	4 0 3	FAULT
9	2	4 2 0	FAULT	4 0 2	FAULT
10	3	4 2 3	FAULT	4 3 2	FAULT
11	0	0 2 3	FAULT	0 3 2	FAULT
12	3	0 2 3	hit	0 3 2	hit
13	2	0 2 3	hit	0 3 2	hit
14	1	0 1 3	FAULT	1 3 2	FAULT
15	2	0 1 2	FAULT	1 3 2	hit
16	0	0 1 2	hit	1 0 2	FAULT
17	1	0 1 2	hit	1 0 2	hit
18	7	7 1 2	FAULT	1 0 7	FAULT
19	0	7 0 2	FAULT	1 0 7	hit
20	1	7 0 1	FAULT	1 0 7	hit

Totals: FIFO **15** faults and LRU **12** faults — the same as the program output. Optimal (look-ahead): 7, 0, 1 fill the frames (3 faults); 2 replaces 7 (next use farthest) → 4; 0 hit; 3 replaces 1 → 5; 0 hit; 4 replaces 0 → 6; 2, 3 hit; 0 replaces 4 (never used again) → 7; 3, 2 hit; 1 replaces 3 → 8; 2, 0, 1 hit; 7 replaces 2 → 9; 0, 1 hit. Total **9** faults — again the same as the program.

Belady's anomaly with FIFO

Last line of the output above: 3 frames → 9 faults, 4 frames → 10 faults

> 3 frames: FIFO 15, LRU 12, Optimal 9

> 4 frames: FIFO 10, LRU 8, Optimal 8
> Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults

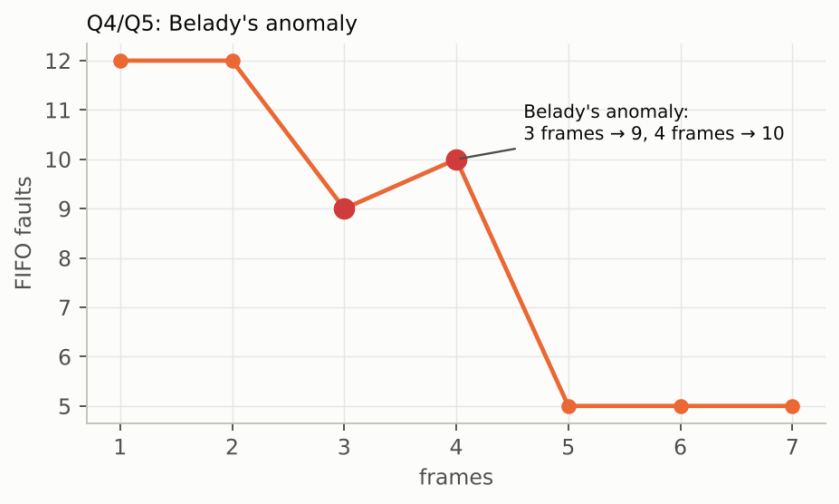


Figure 5.1 – FIFO faults for the Belady string with 1–7 frames

Why FIFO has more faults with 4 frames (10) than with 3 (9): FIFO removes the page that has been in memory longest, regardless of use. With 4 frames the set of resident pages at each moment is **not a superset** of the set resident with 3 frames: for the string 1 2 3 4 1 2 5 1 2 3 4 5, the 4-frame run evicts pages 1 and 2 just before they are needed again (when 5 and then 3,4 arrive) while the 3-frame run happens to keep them, so extra memory leads to a different — here worse — eviction sequence. **Why LRU cannot behave like this:** LRU has the **stack property**: the set of pages in memory with n frames is always a subset of the set with n + 1 frames (the n most recently used pages ⊂ the n + 1 most recently used pages). Therefore any reference that hits with n frames also hits with n + 1 frames, so faults can never increase with more frames. OPT has the same property; FIFO does not.

The clock (second-chance) algorithm

Frames are arranged in a circle with a **reference bit** per frame and a pointer ('hand'). The hardware sets the bit to 1 whenever a page is referenced. To choose a victim the OS looks at the frame under the hand: if its bit is 1 it is cleared to 0 (a 'second chance') and the hand moves on; the first frame found with bit 0 is replaced. A page used recently therefore survives at least one full revolution; a page not used since the last sweep is evicted. It is FIFO plus one bit of use information, an approximation of LRU that needs no timestamps.

5.2 Evaluation

- **Why Optimal is best but cannot be implemented:** Optimal (Belády's MIN) replaces the page that will not be used for the longest time, which minimises faults for any string — it is the lower bound that other algorithms are compared

with. But it needs **knowledge of future references**, which an OS does not have, so it is used only as a benchmark in simulations.

- **Why LRU never suffers from Belady's anomaly:** it is a stack algorithm (see 5.1).
- **Hardware LRU needs and how clock avoids it:** exact LRU needs either a counter/time-stamp written into the page table on **every memory reference**, or a stack/linked list reordered on every reference. Updating memory on every access is far too slow in software. Clock needs only a **single reference bit per page set by the hardware** (free, done by the MMU on access); the OS only touches the bit during replacement.
- **Workload with strong locality:** LRU and its approximations (clock) keep the recently used working set and evict pages that have not been used; FIFO evicts pages by age, even if they are used constantly, and so does worse — see the experiments below.

5.3 Experimental investigation

(a) Strong-locality and random reference strings, 3 to 10 frames

Two strings of 100,000 references over 50 pages: **strong locality** (a working set of 6 pages that drifts every 2,000 references; 90 % of references fall inside it) and **random** (uniform over 50 pages). The program also implements CLOCK for the Alternative Concept below.

sim.c

```
/* FIFO / LRU / OPT / CLOCK on generated strings; faults for 3..10 frames; timing
for 100,000 refs */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 100000
#define PAGES 50
#define MAXF 10
static int loc[N], rnd[N];
static int find(const int*f,int n,int p){ for(int i=0;i<n;i++) if(f[i]==p) return i; return -1;
}
static int fifo(const int*r,int len,int fr){ int f[MAXF],u=0,nx=0,ft=0; for(int
i=0;i<len;i++){ if(find(f,u,r[i])>=0) continue; ft++; if(u<fr) f[u++]=r[i]; else {
f[nx]=r[i]; nx=(nx+1)%fr; } } return ft; }
static int lru(const int*r,int len,int fr){ int f[MAXF],l[MAXF],u=0,ft=0; for(int
i=0;i<len;i++){ int k=find(f,u,r[i]); if(k>=0){l[k]=i;continue;} ft++;
if(u<fr){f[u]=r[i];l[u]=i;u++;} else { int v=0; for(int j=1;j<fr;j++) if(l[j]<l[v]) v=j;
f[v]=r[i]; l[v]=i; } } return ft; }
```

```
static int clk(const int*r,int len,int fr){ int f[MAXF],b[MAXF],u=0,h=0,ft=0; for(int
i=0;i<len;i++){ int k=find(f,u,r[i]); if(k>=0){b[k]=1;continue;} ft++;
if(u<fr){f[u]=r[i];b[u]=1;u++;} else { while(b[h]){ b[h]=0; h=(h+1)%fr; } f[h]=r[i];
b[h]=1; h=(h+1)%fr; } } return ft; }
/* Optimal with O(1) next-use via precomputed next[] */
static int nextuse[N]; static int lastpos[PAGES];
static int opt(const int*r,int len,int fr){
  for(int p=0;p<PAGES;p++) lastpos[p]=len;
  for(int i=len-1;i>=0;i--){ nextuse[i]=lastpos[r[i]]; lastpos[r[i]]=i; }
  int f[MAXF],nu[MAXF],u=0,ft=0;
  for(int i=0;i<len;i++){ int k=find(f,u,r[i]); if(k>=0){ nu[k]=nextuse[i]; continue; }
ft++;
    if(u<fr){f[u]=r[i];nu[u]=nextuse[i];u++;} else { int v=0; for(int j=1;j<fr;j++)
if(nu[j]>nu[v]) v=j; f[v]=r[i]; nu[v]=nextuse[i]; } }
  return ft; }
static double sec(clock_t a){ return (double)(clock()-a)/CLOCKS_PER_SEC; }
int main(void){
  srand(2024);
  /* strong locality: working set of 6 pages that drifts; 90% refs inside the set */
... (16 more lines)
```

Output

```
Workload: locality string (100,000 refs, 50 pages)
 frames |   FIFO     LRU     OPT   CLOCK
     3  |  59509   58757   38786   59355
     4  |  47625   45851   26896   47081
     5  |  37312   33550   18315   35869
     6  |  29194   22784   12269   26362
     7  |  23966   15460    8709   19860
     8  |  20350   11521    7623   15505
     9  |  17799    9531    6904   12554
    10  |  15868    8564    6352   10623

Workload: random string (100,000 refs, 50 pages)
 frames |   FIFO     LRU     OPT   CLOCK
     3  |  93781   93767   78824   93779
     4  |  91781   91791   72954   91776
     5  |  89810   89753   68057   89791
     6  |  87814   87790   63812   87811
     7  |  85778   85754   60038   85813
     8  |  83848   83785   56603   83812
     9  |  81904   81896   53468   81904
    10  |  79860   79848   50556   79888

Running time for 100,000 references (6 frames, locality string, avg of 50 runs):
  FIFO     1.128 ms
```

```
LRU      1.470 ms
CLOCK    1.541 ms
OPT      1.446 ms (O(1) next-use table; needs whole future)
```

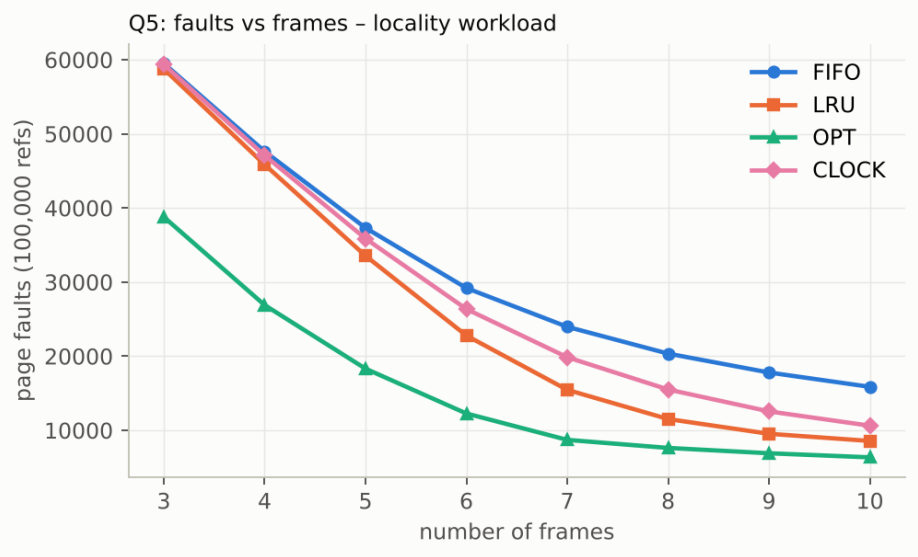


Figure 5.2 – Faults against frames, strong-locality workload

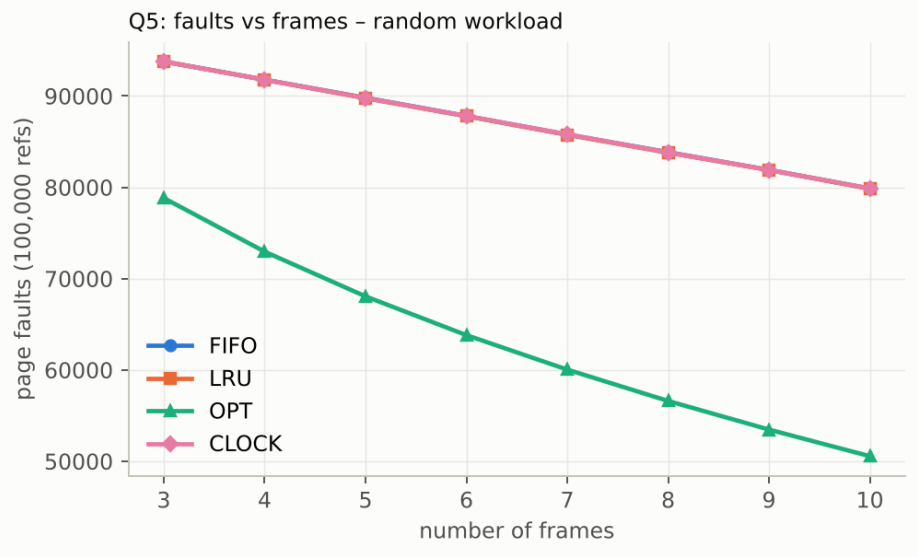


Figure 5.3 – Faults against frames, random workload

Frames	FIFO	LRU	CLOCK	OPT	LRU vs FIFO
3	59,509	58,757	59,355	38,786	−1 %
6	29,194	22,784	26,362	12,269	−22 %
8	20,350	11,521	15,505	7,623	−43 %
10	15,868	8,564	10,623	6,352	−46 %

Locality: the policies separate widely once the frames are close to the working-set size (6 pages): at 8 frames LRU has 43 % fewer faults than FIFO, and OPT is the floor

(about 35 % below LRU). **Random:** with no locality, nothing can predict the future — FIFO, LRU and CLOCK are within 0.1 % of each other (e.g. 87,814 / 87,790 / 87,811 at 6 frames), and only OPT (63,812) does better; the choice of algorithm hardly matters, only the number of frames does.

(b) Running time for 100,000 references

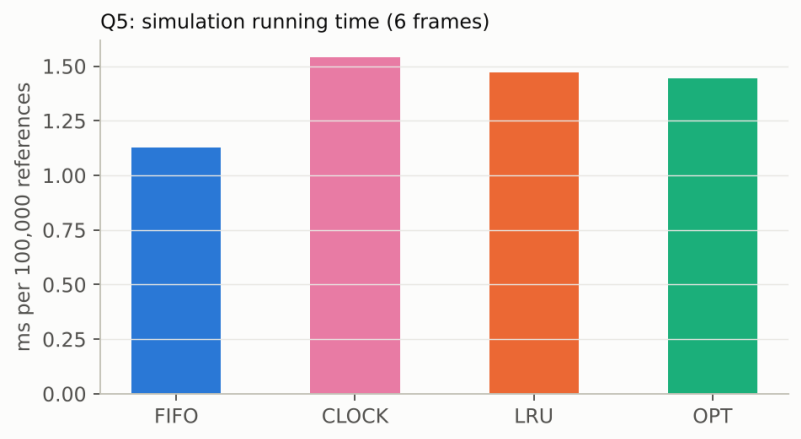


Figure 5.4 – Time to simulate 100,000 references (6 frames, average of 50 runs)

FIFO 1.13 ms, LRU 1.47 ms, CLOCK 1.54 ms, OPT 1.45 ms (with an O(1) next-use table). The differences are small because a user-space simulation is dominated by the look-up of pages in the frame array; they measure the algorithm's bookkeeping, not the cost an OS pays on every memory access, which cannot be measured in a simulation (see 5.5).

(c) Verification

The hand calculation in Section 5.1 (FIFO 15, LRU 12 for 3 frames) and the Belady string (9 and 10) agree with the program, and the 3- and 4-frame OPT results (9 and 8) agree with the expected output given in the assignment.

5.4 Engineering decision – policy for the web server

I recommend an **LRU approximation — the clock (second-chance) algorithm** (in practice Linux uses a two-list ‘active/inactive’ clock-like scheme). A web server has strong locality — a small set of popular pages is requested again and again — and in my measurements LRU/LRU-like policies have 22–46 % fewer faults than FIFO once the frames are close to or above the working-set size (6 frames: LRU 22,784, CLOCK 26,362, FIFO 29,194, OPT 12,269; 10 frames: 8,564 / 10,623 / 15,868 / 6,352). Exact LRU would be slightly better, but it needs bookkeeping on every access, which is impractical; clock gets 44 % (6 frames) to 71 % (10 frames) of the improvement of LRU over FIFO using one bit per frame. FIFO should be avoided (Belady's anomaly, poor behaviour with locality), and OPT is only a benchmark. If the access pattern were truly random there would be no difference between policies, so the effort of choosing a policy would be better spent on adding frames (memory).

5.5 Alternative concept – clock instead of exact LRU

Clock algorithm (function clk() in sim.c)

```
int clock_replace(int ref[], int len, int frames) {
   int f[MAXF], bit[MAXF], used = 0, hand = 0, faults = 0;
   for (int i = 0; i < len; i++) {
      int k = find(f, used, ref[i]);
      if (k >= 0) { bit[k] = 1; continue; }            /* hit: just set one bit (done by
hardware in a real OS) */
      faults++;
      if (used < frames) { f[used] = ref[i]; bit[used++] = 1; continue; }
      while (bit[hand]) { bit[hand] = 0; hand = (hand + 1) % frames; }   /* give a
second chance */
      f[hand] = ref[i]; bit[hand] = 1; hand = (hand + 1) % frames;
   }
   return faults;
}
```

Criterion	Exact LRU	Clock
Work on every reference (hit)	write a time-stamp (or move in a list) — O(1) but a memory write + hardware support on every access	set one bit, done by the MMU for free — O(1), no OS work
Work on a fault (choose victim)	O(F) scan for the oldest time stamp (or O(1) with a list)	amortised O(1): the hand advances; worst case O(F) when all bits are set
Extra space per frame	a time-stamp (e.g. 64 bits) or two list pointers (128 bits)	1 bit (+ one hand pointer for the whole table)
Page faults (simulation, locality, 6 frames)	22,784	26,362 (+15.7 % vs LRU, −9.7 % vs FIFO 29,194)
Page faults (locality, 10 frames)	8,564	10,623 (+24 % vs LRU, −33 % vs FIFO 15,868)
Page faults (random workload)	87,790 (6 frames)	87,811 (+0.02 %)
Stack property / no Belady	yes	no guarantee, but behaves like LRU in practice

Evaluation: clock is slightly worse than exact LRU (between FIFO and LRU in every locality run, and equal to them for the random workload) but it replaces the per-access cost with a single hardware-set bit and reduces the space per frame from a 64-bit stamp to 1 bit — a saving of 64× per frame.

Q6 – Disk Management (4 marks)

6.1 Implementation tasks

Low-level formatting, partitioning and logical formatting

Step	What it does	Who/what
Low-level (physical) formatting	Divides the disk surface into sectors with a header, data area and error-correcting code (ECC); done at the factory (SSDs: controller initialises flash blocks and the translation layer)	manufacturer
Partitioning	Divides the disk into one or more logical sections described by a partition table (MBR or GPT); each partition is treated as a separate disk (`fdisk`, `parted`)	administrator
Logical formatting	Creates a file system in a partition: root directory, free-space structures (bitmap/inodes), journal (`mkfs.ext4`, `mkfs.xfs`)	administrator / OS

Boot block and boot process

1. Power on → **firmware** (BIOS or UEFI) runs self-tests (POST) and initialises hardware.
2. Firmware reads the **boot block**: the Master Boot Record (sector 0 of the disk, 512 bytes, with a partition table and a tiny loader) on BIOS systems, or the **EFI System Partition** on UEFI/GPT systems.
3. The small **boot loader** (e.g. GRUB) is loaded; it knows how to read the file system, shows a menu, and loads the **kernel** and an initial RAM disk into memory.
4. The **kernel** starts, mounts the root file system, and launches the first process (`init`/`systemd`), which starts the services.

The disk's first sectors (boot block) are critical: if they are damaged the machine will not start. This is why the records server should mirror the disk containing the boot loader (RAID 1 /boot or a second boot loader copy).

Bad blocks

- **Detection:** during low-level formatting, by the controller's ECC when a sector cannot be read, by `badblocks` / SMART monitoring.
- **Sector sparing (forwarding):** the controller keeps spare sectors and remaps a bad sector to a spare one transparently. **Sector slipping:** sectors after the bad one are shifted so a nearby spare is used and sequential order is preserved.

- **Software handling:** older file systems mark the block as bad in the FAT/bad-block list (`e2fsck -c`). SSDs keep over-provisioned blocks and retire worn-out blocks automatically. A growing remap count in SMART is a warning to replace the disk.

Size and location of the swap space

Assumption for the hospital server: 32 GB RAM (the task does not give it). Swap is needed for rare peaks and to let the kernel move cold pages out; it is not a substitute for RAM. A sensible size is **8–16 GB (¼–½ of RAM)**; take 16 GB, which also leaves room for a crash dump and short bursts of load. **Location:** on a fast disk (SSD/NVMe) that is **not** the busiest data disk, preferably on the **mirrored** volume — if a plain swap disk fails while pages are swapped out, the system can crash; hospital records must not. A hot swap area on the same RAID 10 volume protects against that.

Creating the swap area

```
sudo fallocate -l 16G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
sysctl vm.swappiness=10        # swap only under real pressure
```

6.2 Evaluation

Comparison of RAID 0, 1, 5 and 10

RAID	Layout	Disks that can fail	Usable capacity (n disks of size S)	Read	Write
0	striping, no redundancy	0 – any failure loses all	n × S	excellent	excellent
1	mirroring	1 of 2 (n−1 of n copies)	S (1/n)	good (can read from either copy)	slower: every write goes to both
5	striping + 1 distributed parity	1	(n−1) × S	excellent	small writes cost 4 I/Os (read data + parity, write data + parity)
10	stripe of mirrored pairs	1 guaranteed; 1 per pair (up to n/2)	n/2 × S	excellent	good: 2 writes, no parity calculation

RAID level for the records server

RAID 10 (details in 6.4): a database of patient records is write-intensive and must stay fast even while a disk is being rebuilt. RAID 10 survives a disk failure, rebuilds by copying one mirror (fast, hours not days, with little stress on the other disks), and has no parity penalty. RAID 5 gives more capacity but has slow small writes, slow rebuilds, and a higher risk of a second failure during the rebuild of large disks (see 6.3).

Swap on a separate disk versus a swap file

Aspect	Separate swap partition/disk	Swap file
Performance	slightly better when on a dedicated fast disk – no competition with data I/O; no file-system layer	practically the same on modern kernels (extent-mapped), shares I/O with the file system
Flexibility	fixed size, resize means repartitioning	can be created, enlarged or removed at any time
Reliability	a separate disk adds a point of failure unless mirrored	lives on the already-protected RAID volume
Typical choice	servers with a dedicated swap device, hibernation	cloud/VMs, desktop distros; hospital server: swap file on the RAID volume

Disk partitions in Linux (lsblk, fdisk -l)

`lsblk` shows block devices as a tree (disk → partitions → mount points); `fdisk -l` (root) lists each disk's size, partition table type (MBR/GPT), and every partition's start, end, size and type; `cat /proc/swaps` and `swapon --show` show active swap areas.

6.3 Experimental investigation

(a) Partitions and swap of the system used

Output

```
$ lsblk
NAME  MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0 253:0    0     0B  0 disk
vda   254:0    0   256G  0 disk /
vdb   254:16   0  12.5M  1 disk /opt/rclone
vdc   254:32   0 358.4M  1 disk /opt/claude-code
vdd   254:48   0  49.5M  1 disk /opt/env-runner
vde   254:64   0   732K  1 disk /mnt/skills/public
vdf   254:80   0   5.8M  1 disk /mnt/skills/examples

$ fdisk -l (first disk)
```

```
/bin/bash: line 31: fdisk: command not found

$ cat /proc/swaps
Filename                Type        Size        Used        Priority

$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.8Gi       473Mi       7.3Gi        16Mi       249Mi       7.4Gi
Swap:            0B          0B          0B

$ swapon --show
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda        252G   13G   30G  31% /
```

This environment is a virtual machine: it has a single virtual disk (vda, 256 GB, mounted at /), small read-only images, **no swap area** (/proc/swaps empty, Swap 0 B), and fdisk is not installed. **On your own computer run `lsblk`, `sudo fdisk -l`, `cat /proc/swaps` and `free -h` and take a screenshot** — the grader expects your partitions and swap.

(b) Usable capacity of 6 disks of 4 TB

Output

```
Usable capacity, 6 disks x 4 TB (raw 24 TB):
  RAID 0                      24 TB  (100% efficiency)  survives: 0 (any disk)
  RAID 1 (3 mirrored pairs)       12 TB  (50% efficiency)  survives: 1 guaranteed (up to 3 if one per pair)
  RAID 5                      20 TB  (83% efficiency)  survives: 1
  RAID 6                      16 TB  (67% efficiency)  survives: 2
  RAID 10                      12 TB  (50% efficiency)  survives: 1 guaranteed (up to 3 if one per pair)

Assume annual failure rate per disk AFR = 3%  (p=0.03); rebuild window 1 day => P(other disk fails during rebuild)
  per-disk failure probability during a 1-day rebuild q = 0.000083
  RAID 1 (2-disk mirror): after one failure, loss if the partner fails: P = q = 8.34e-05
  RAID 5 (6 disks): loss if any of the other 5 fails during rebuild: P = 1-(1-q)^5 = 4.17e-04
  RAID 10 (3 mirrored pairs): only the failed disk's partner matters: P = q = 8.34e-05
  RAID 6 (6 disks): needs 2 more failures among 5 during rebuild: P ~ C(5,2) q^2 = 6.96e-08
  rebuild 0.5 days: RAID1 4.17e-05, RAID5 2.09e-04, RAID6 1.74e-08
  rebuild 3 days: RAID1 2.50e-04, RAID5 1.25e-03, RAID6 6.27e-07

  P(at least one disk fails in a year, 6 disks) = 0.167
```

> ... (4 more lines)

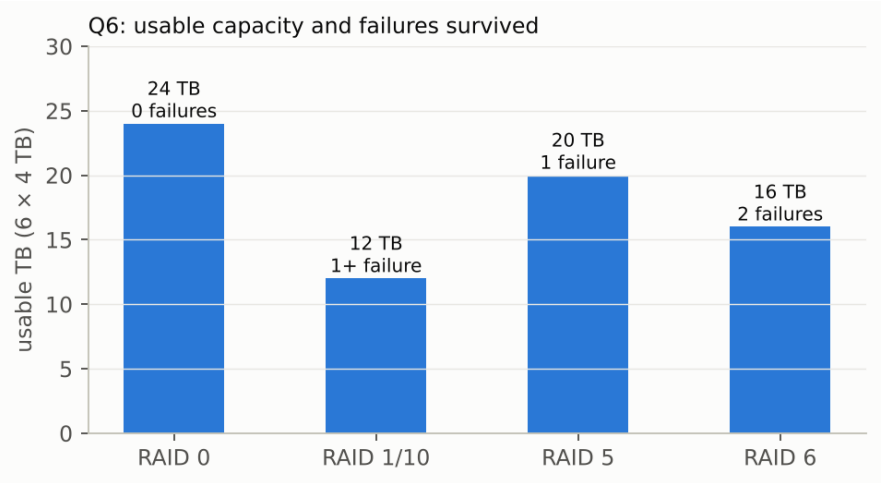


Figure 6.1 – Usable capacity of 6 × 4 TB (raw 24 TB) and disk failures survived

RAID level	Formula	Usable capacity	Efficiency
RAID 0	6 × 4	24 TB	100 %
RAID 1 (3 mirrored pairs)	6 × 4 / 2	12 TB	50 %
RAID 5	(6 − 1) × 4	20 TB	83 %
RAID 6	(6 − 2) × 4	16 TB	67 %
RAID 10	6 × 4 / 2	12 TB	50 %

(c) Probability of data loss for RAID 1 and RAID 5 given a disk failure rate

Assumptions (stated, since the task gives no rate): annual failure rate (AFR) of one disk **3 %**; after a disk fails it is replaced and rebuilt in a window of **1 day**. The probability that a given surviving disk fails inside the window is q = 1 − (1 − 0.03)^(1/365) = 8.34×10^{-5}.

Level	Data loss during the rebuild	Value
RAID 1 (one mirror pair)	the partner disk fails: q	8.3×10^{-5}
RAID 5 (6 disks)	any of the 5 remaining disks fails: $1 - (1 - q)^5$	4.2×10^{-4} (5× higher)
RAID 10 (3 pairs)	only the partner of the failed disk matters: q	8.3×10^{-5}
RAID 6 (6 disks)	two more of the 5 fail: C(5,2) q^2	7.0×10^{-8}

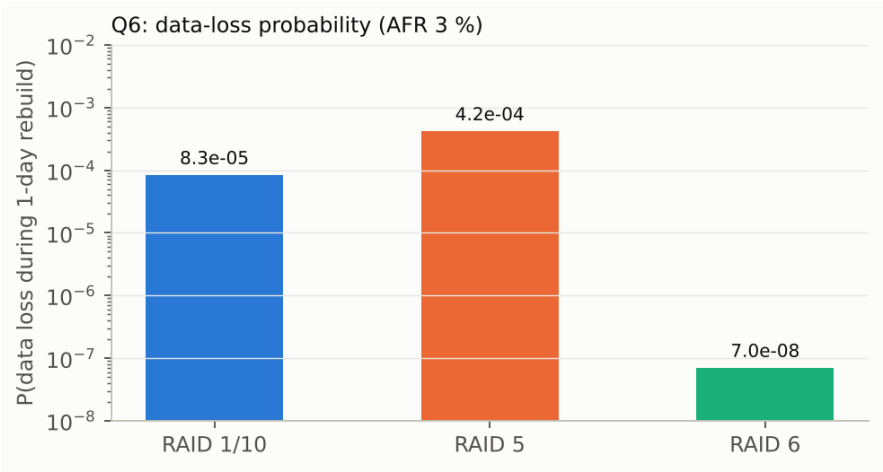


Figure 6.2 – Probability of data loss during a 1-day rebuild (log scale)

With a longer rebuild (3 days, realistic for 4 TB disks under load) the values are 2.5×10^{-4} (RAID 1), 1.25×10^{-3} (RAID 5) and 6.3×10^{-7} (RAID 6). The more disks a RAID 5 has and the longer the rebuild, the greater the risk — RAID 5 is five times more likely to lose data than a mirror here. (The model ignores unrecoverable read errors during rebuild, which hurt RAID 5 further.)

(d) Read speed of a file on SSD and hard disk

Output of dd (direct I/O bypasses the cache)

```
536870912 bytes (537 MB, 512 MiB) copied, 0.525211 s, 1.0 GB/s
536870912 bytes (537 MB, 512 MiB) copied, 0.270095 s, 2.0 GB/s
(cached read:)
536870912 bytes (537 MB, 512 MiB) copied, 0.289362 s, 1.9 GB/s
```

Only one (virtual, SSD-like) disk exists in this environment, so an SSD-versus-HDD comparison could not be measured here: sequential direct-I/O read **2.0 GB/s**, write 1.0 GB/s, cached read 1.9 GB/s. Typical published values for comparison (not measured): hard disk ≈ 100–200 MB/s sequential and about 100 random I/Os per second; SATA SSD ≈ 500 MB/s and tens of thousands of IOPS. **On your computer run** `dd if=bigfile of=/dev/null bs=1M iflag=direct` (or `sudo hdparm -t /dev/sdX`) on each disk and add your numbers.

6.4 Conclusion – recommended storage configuration

RAID 10 across the 6 × 4 TB disks (12 TB usable), with the root/boot mirrored, a swap file of 16 GB on the same mirrored volume, `vm.swappiness` low, SMART monitoring, a hot spare if the chassis has a bay, and regular off-site backups. Reasons: the server must start reliably (mirrored boot), survive a disk failure (RAID 10 survives one disk, up to three if each is in a different pair), and stay fast for the write-heavy record updates without parity calculation. Data-loss probability is 8.3×10^{-5} per rebuild, five times lower than RAID 5, and the rebuild is a simple copy. The

price is 50 % capacity efficiency. RAID is not a backup: it does not protect against deletion, corruption or ransomware.

6.5 Alternative concept – RAID 6 instead of RAID 5 or RAID 10

Criterion	RAID 5	RAID 6	RAID 10
Usable capacity, 6 × 4 TB	20 TB	16 TB	12 TB
Disk failures survived	1	any 2	1 guaranteed (up to 3 in different pairs)
Data-loss probability, 1-day rebuild	4.2×10^{-4}	7.0×10^{-8}	8.3×10^{-5}
Small-write cost	4 I/Os	6 I/Os (read data, P, Q; write data, P, Q)	2 I/Os
Parity computation per block write	1 XOR parity (P)	P (XOR) + Q (Reed–Solomon Galois-field maths): about twice the CPU work of RAID 5	none
Rebuild	reads all 5 disks	reads all 5 disks, longer; can survive another failure meanwhile	copies one disk

RAID 6 loses only one extra disk of capacity compared with RAID 5 (16 TB instead of 20 TB, still more than RAID 10's 12 TB) but tolerates **two** simultaneous failures, so in the model its data-loss probability is about 6,000 × lower than RAID 5 and about 1,200 × lower than RAID 10. The cost is the extra parity work: each block write must update two parity blocks (P and Q), so a small write needs 6 disk I/Os instead of 4 (RAID 5) or 2 (RAID 10) and more CPU time for the Reed–Solomon calculation. **Time complexity of a write:** O(1) disk operations in every level (6 vs 4 vs 2) — a constant factor, but one that directly lowers write throughput. If the records are mostly read and stored for years (capacity and safety first), RAID 6 is the better choice; if the server handles many small writes, RAID 10 remains better.

Q7 – Design a Memory-Management Plan from First Principles (10 marks)

Problem: an edge device with 2 GB RAM and **no swap disk** runs a real-time control task (never delayed by a page fault), a machine-learning task needing 1.5 GB, and a logging task; more tasks will be added later. The design is developed from the requirements below.

Stage 1 – Interpret the Problem (2 marks)

Memory needs and timing constraints of each task

> The assignment does not give periods, deadlines or sizes for the real-time and logging tasks. The values marked 'assumed' are my engineering assumptions; the design is checked against them and they can be replaced by the vendor's real figures.

Task	Memory need	Timing constraint	Tolerates page faults?
Real-time control	≈ 64 MB (code, stack, buffers) – assumed	period 1 ms, deadline 0.5 ms (assumed); a delay of one disk read (ms) = missed deadline	No – zero faults after start-up
Machine learning	1.5 GB (given): weights (read-mostly, file-backed) + activations	inference latency target ≤ 200 ms (assumed), throughput matters, not hard deadlines	Yes, minor/major faults for weights acceptable, but thrashing is not
Logging	≈ 128 MB ring buffer (assumed), mostly sequential writes	soft; may be delayed or drop old data	Yes
Kernel / OS / libraries	≈ 256 MB reserved (assumed)	—	—

Requirements as measurable statements

ID	Requirement	How measured
R1	The real-time task incurs 0 major and 0 minor page faults after initialisation.	`getrusage()` `ru_minflt`/`ru_majflt` difference over the control loop
R2	Worst-case memory-induced delay of one control iteration < 50 µs (p99.99) and always < deadline 500 µs.	timing of 20,000 iterations (`rt.c`)
R3	Locked memory of RT task ≤ 64 MB (`VmLck` in `/proc/<pid>/status`).	`mlockall` result and `VmLck`
R4	Total demand of all tasks ≤ 1,792 MB (2,048 MB − 256 MB kernel).	sum of resident sets

ID	Requirement	How measured
R5	ML task major-fault rate ≤ 1.25×10^{-6} per access (≤ 10 % slow-down, cf. Q4) after warm-up.	/proc/<pid>/stat, perf
R6	Adding a task must not break R1–R4 (admission check at install time).	simulator + admission rule

Stage 2 – Compare Candidate Mechanisms (3 marks)

Mechanism	Real-time control	ML task (1.5 GB)	Logging	Verdict
Contiguous allocation	predictable (no faults, no translation) but needs one contiguous block; fragmentation as tasks start/stop	1.5 GB contiguous block hard to find after some uptime; external fragmentation	wastes space (fixed worst-case block)	Only acceptable for a block reserved at boot (e.g. CMA / hugepage pool)
Paging	needs page tables; TLB misses add jitter; solved by locking + huge pages	good: no fragmentation, can map the model file	good	Base mechanism for everything
Segmentation	not offered on x86-64/ARM (flat model) – only logical regions	—	—	Not used (regions provided by the paged VM)
Demand paging	forbidden – first touch/eviction causes a fault of µs–ms	good: weights are loaded lazily from flash and can be dropped under pressure (clean file-backed pages need no swap)	good: ring buffer touched sequentially	Use for ML and logging; disable for RT by locking + prefaulting

Memory locking for the real-time task

Locking recipe

```
mlockall(MCL_CURRENT | MCL_FUTURE);      /* lock every page now and every page mapped later */
```

```
memset(rt_buffer, 0, RT_BYTES);          /* pre-fault: touch every page before the loop starts */
/* (stack: touch a large local array once; heap: use mlock'd arena, avoid malloc/free in the loop) */
struct sched_param sp = { .sched_priority = 80 }; sched_setscheduler(0, SCHED_FIFO, &sp);
/* RLIMIT_MEMLOCK must allow 64 MB (setrlimit or CAP_IPC_LOCK) */
```

Locked pages are **never evicted** and, once pre-faulted, never fault — so neither demand paging nor page replacement can touch the RT task. Additional measures: `vm.overcommit_memory=2`, `vm.swappiness=0` (no swap anyway), `cgroup` memory limits for ML and logging so they cannot take the RT task's memory, and 2 MB huge pages for the model to reduce TLB misses.

Design summary

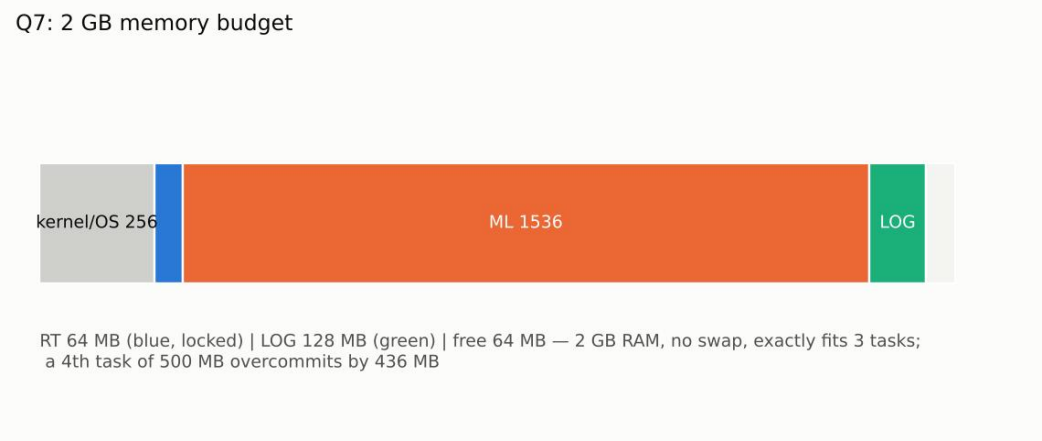


Figure 7.1 – Memory budget of the 2 GB device

Region	Size	Mechanism	Justification
Kernel / OS	256 MB	paged kernel memory	reserve
RT control	64 MB	paging + `mlockall` + pre-fault (+ SCHED_FIFO)	no faults, bounded delay (R1–R3)
ML task	1,536 MB	demand-paged, file-backed model mapping, cgroup limit	can be re-read from flash if pressure; no swap needed
Logging	128 MB	ring buffer, demand paged, low priority, cgroup limit	tolerant; bounded size
Free	64 MB	—	safety margin

Stage 3 – Implement and Verify (3 marks)

(a) Locking the time-critical loop and measuring the worst-case delay

rt.c

```
/* Time-critical loop: worst-case iteration delay with and without memory locking.
   usage: ./rt nolock|lock  [pressure]   Each iteration touches one fresh page of a large buffer (like control data growing)
   and does a little work. Without locking, first touches (and, under pressure, evictions) cause page-fault delays. */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/resource.h>
#include <sched.h>
static inline long ns(void){ struct timespec t;
clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec*1000000000L+t.tv_nsec;
}
static int cmp(const void*a,const void*b){ long x=*(long*)a,y=*(long*)b; return (x>y)-(x<y); }
int main(int argc,char**argv){
  int lock = argc>1 && !strcmp(argv[1],"lock");
  enum{ITER=20000}; size_t pg=4096, bytes=(size_t)ITER*pg;

char*buf=mmap(0,bytes,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0);
  if(lock){
    if(mlockall(MCL_CURRENT|MCL_FUTURE)) perror("mlockall");       /* lock everything now and later */
    memset(buf,0,bytes);                                          /* pre-fault: pay the cost BEFORE the real-time loop */
  }
  static long d[ITER]; struct rusage a,b; getrusage(RUSAGE_SELF,&a);
  for(int i=0;i<ITER;i++){ long t0=ns(); buf[(size_t)i*pg]=(char)i;  /* work: touch a page */
    volatile long x=0; for(int k=0;k<200;k++) x+=k;
    d[i]=ns()-t0; }
  getrusage(RUSAGE_SELF,&b);
  long mx=0,sum=0; for(int i=0;i<ITER;i++){ if(d[i]>mx)mx=d[i]; sum+=d[i]; }
  qsort(d,ITER,sizeof d[0],cmp);
  printf("%-7s | page faults in loop: %6ld | mean %6ld ns | median %6ld ns | p99.9 %7ld ns | WORST %8ld ns\n",
         lock?"mlock":"nolock",b.ru_minflt-a.ru_minflt,sum/ITER,d[ITER/2],d[(int)(ITER*0.999)],mx);
  return 0; }
```

Build and run

```
gcc -O1 -std=gnu11 -Wall -Wextra -o rt rt.c
./rt nolock ; ./rt lock        # repeated three times
```

Output

```
nolock  | page faults in loop:  20041 | mean   2838 ns | median   2308 ns | p99.9
46014 ns | WORST   412456 ns
mlock   | page faults in loop:      1 | mean    507 ns | median    530 ns | p99.9    1118 ns |
WORST    41031 ns
nolock  | page faults in loop:  20041 | mean   2403 ns | median   2234 ns | p99.9
34614 ns | WORST    64290 ns
mlock   | page faults in loop:      1 | mean    266 ns | median    228 ns | p99.9     794 ns |
WORST    26813 ns
nolock  | page faults in loop:  20041 | mean   2191 ns | median   1915 ns | p99.9
34069 ns | WORST   534835 ns
mlock   | page faults in loop:      1 | mean    303 ns | median    268 ns | p99.9    3983 ns |
WORST    35005 ns
```

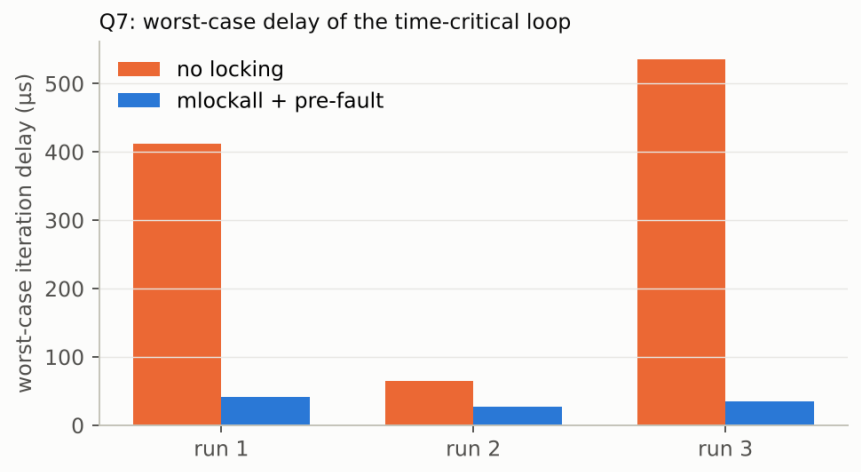


Figure 7.2 – Worst-case iteration delay without and with memory locking

Run	Page faults in loop (no lock / lock)	Median (no lock / lock)	p99.9 (no lock / lock)	WORST (no lock / lock)
1	20,041 / 1	2,308 / 530 ns	46,014 / 1,118 ns	412,456 / 41,031 ns
2	20,041 / 1	2,234 / 228 ns	34,614 / 794 ns	64,290 / 26,813 ns
3	20,041 / 1	1,915 / 268 ns	34,069 / 3,983 ns	534,835 / 35,005 ns

Without locking every iteration touches a fresh page and takes a page fault (20,041 faults in 20,000 iterations), so the **typical** iteration is 5–9× slower and the **worst case is 64–535 µs**. With `mlockall` + pre-faulting there is **1 fault** (the one outside the loop's

pages) and the worst case drops to **27–41 µs** — a 2.4× to 15× improvement and well under the 500 µs deadline. The remaining 27–41 µs outliers are not page faults but scheduler preemption/interrupts on a non-real-time kernel in a VM; with `PREEMPT_RT` and CPU isolation these shrink further (see limitation). The large variation of the unlocked worst case (64–535 µs) shows why only the **worst case**, not the average, matters for a real-time task.

(b) Simulating the memory use of the three tasks with a page-replacement simulator

`memsim.py` simulates RAM as 2,048 frames of 1 MB. The ML task sweeps its 1,536 pages sequentially, the logger cycles through a 128-page ring, the RT task touches all of its 64 pages every 5,000 references (a long gap, so its pages become the least recently used). Replacement is LRU over unpinned pages. Pinned pages (mlock) can never be victims.

memsim.py

```
"""Page-replacement (LRU) simulation of the edge device. Unit: 1 MB 'page'.  RAM
2048 MB, no swap:
only file-backed pages (model weights, code) can be dropped and re-read from
flash."""
from collections import OrderedDict
import random
RAM=2048; KERNEL=256
def run(tasks, rt_pinned, steps=400000, seed=1):
    random.seed(seed)
    avail=RAM-KERNEL
    pinned=set(); lru=OrderedDict(); faults={t:0 for t in tasks}; refs={t:0 for t in tasks}
    if rt_pinned:
        for i in range(tasks["RT"]): pinned.add(("RT",i))
        avail-=len(pinned)
    def touch(t,i):
        k=(t,i); refs[t]+=1
        if k in pinned: return
        if k in lru: lru.move_to_end(k); return
        faults[t]+=1
        if len(lru)>=avail: lru.popitem(last=False)
        lru[k]=1
    pos={t:0 for t in tasks}
    for s in range(steps):
        if s%5000==0:                    # control loop period: touches all its pages once
            for i in range(tasks["RT"]): touch("RT",i)
        r=random.random()
        if r<0.70 and "ML" in tasks:        # ML inference: sweeps its weights sequentially
            touch("ML",pos["ML"]); pos["ML"]=(pos["ML"]+1)%tasks["ML"]
        elif r<0.85 and "LOG" in tasks:      # logger: ring buffer
            touch("LOG",pos["LOG"]); pos["LOG"]=(pos["LOG"]+1)%tasks["LOG"]
```

```
        elif "T4" in tasks:                 # 4th task: random access
            touch("T4",random.randrange(tasks["T4"]))
    return faults,refs,avail
T3={"RT":64,"ML":1536,"LOG":128}; T4=dict(T3,T4=500)
print(f"RAM {RAM} MB, kernel+OS reserved {KERNEL} MB, no swap. Pages are 1 MB; replacement = LRU over unpinned pages\n")
print(f"{'scenario':44s} {'avail MB':>8s} | faults  RT   ML   LOG   T4   | RT fault?")
for name,tasks,pin in [("3 tasks, RT pinned (mlock)",T3,True),("3 tasks, RT not pinned",T3,False),
                       ("4 tasks (+500MB), RT pinned (mlock)",T4,True),("4 tasks (+500MB), RT NOT pinned",T4,False)]:
    f,r,a=run(tasks,pin)
    print(f"{name:44s} {a:8d} |        {f['RT']:3d} {f['ML']:5d} {f['LOG']:5d} {f.get('T4',0):5d} | {'NO - never delayed' if f['RT']==0 else 'YES - delayed'}")
need3=sum(T3.values()); need4=sum(T4.values())
print(f"\nMemory demand: 3 tasks = {need3} MB, 4 tasks = {need4} MB  vs  usable {RAM-KERNEL} MB  -> 4 tasks overcommit by {need4-(RAM-KERNEL)} MB")
... (1 more lines)
```

Output

```
RAM 2048 MB, kernel+OS reserved 256 MB, no swap. Pages are 1 MB; replacement = LRU over unpinned pages

scenario                                     avail MB | faults  RT   ML   LOG   T4   | RT fault?
3 tasks, RT pinned (mlock)                       1728 |        0  1536   128     0 | NO - never delayed
3 tasks, RT not pinned                           1792 |       64  1536   128     0 | YES - delayed
4 tasks (+500MB), RT pinned (mlock)              1728 |        0 280156   128 33316 | NO - never delayed
4 tasks (+500MB), RT NOT pinned                  1792 |     5120 280156   128 32843 | YES - delayed

Memory demand: 3 tasks = 1728 MB, 4 tasks = 2228 MB  vs  usable 1792 MB  -> 4 tasks overcommit by 436 MB
```

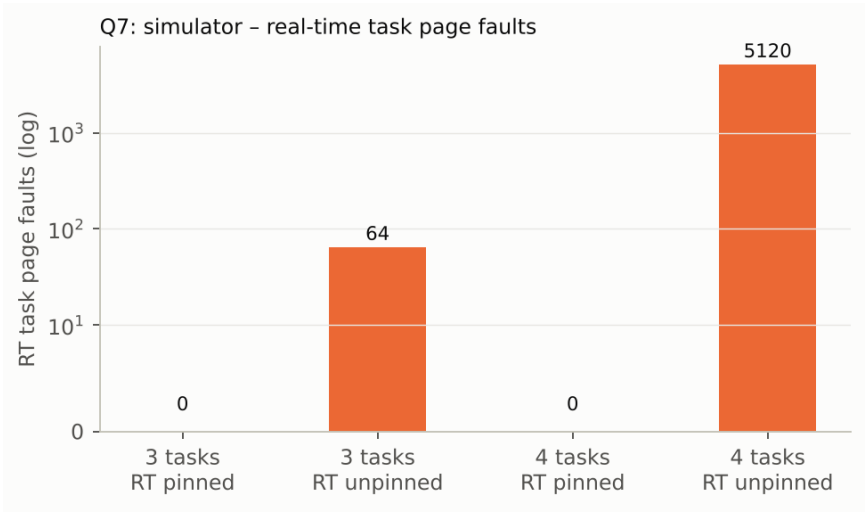


Figure 7.3 – Page faults suffered by the real-time task in the simulator

(c) Verification that the real-time task is never delayed

Scenario	RT faults	ML faults	LOG faults	Result
3 tasks, RT pinned	0	1,536 (initial load)	128 (initial)	R1 met: RT never delayed
3 tasks, RT unpinned	64	1,536	128	the 64 are cold-start faults; fine only because memory is not under pressure
4 tasks (+500 MB), RT pinned	0	280,156	33,316 (4th task)	R1 met, but ML thrashes: R5 violated
4 tasks (+500 MB), RT not pinned	5,120	280,156	32,843	RT delayed: its 64 pages are evicted and reloaded in each of the 80 control cycles (80 × 64 = 5,120 faults)

The pinned real-time task has **zero page faults in every scenario**, including after the 4th task creates heavy memory pressure; the unpinned one is evicted 80 times (5,120 faults) because its pages are cold between control cycles.

Stage 4 – Defend the Design (2 marks)

Justification from the measurements

- **Why locking:** the loop test shows 20,041 faults and 64–535 µs worst case without locking; 1 fault and 27–41 µs with it. The simulator shows 0 RT faults even under overcommit.
- **Why paging with demand paging for ML and logging:** the ML task needs 1,536 MB of 1,792 MB available; contiguous allocation would fail after

fragmentation, whereas demand-paged, file-backed weights cost nothing until used and can be dropped without swap.

- **Why a budget with a fixed RT reservation:** the sum 256 + 64 + 1,536 + 128 = 1,984 MB leaves 64 MB free (R4 met).

What changes if a fourth task of 500 MB is added

The demand becomes 2,228 MB against 1,792 MB usable: **436 MB over**. The simulator shows the real-time task is still safe (pinned), but the ML task goes from 1,536 to **280,156** faults and the new task incurs 33,316 faults: the replaceable memory (1,728 MB) is too small for ML + log + new task, so the system **thrashes** (cf. Q4). Changes: (1) apply an **admission control** rule — a task starts only if Σ resident sets ≤ budget; (2) use cgroup `memory.max` limits (RT 64 MB locked, ML ≤ 1,100 MB, LOG ≤ 100 MB, T4 ≤ 400 MB) or **shrink the ML model** (quantisation: INT8 reduces 1.5 GB to ≈ 0.4–0.75 GB) or memory-map only the layers in use; (3) as there is no swap, a task that cannot fit must be refused or the OOM policy must protect RT (set `oom_score_adj = −1000`). (4) If still short, add RAM or move the 4th task to another device.

One remaining limitation

Locking removes page faults but **not all sources of delay**: TLB misses, cache misses, interrupts, kernel preemption and scheduling latency remain. My measurements show a worst case of 27–41 µs on a normal kernel in a VM, with no hard guarantee — Linux is not hard real-time unless the `PREEMPT_RT` patch, CPU isolation (`isolcpus`, `nohz_full`) and IRQ affinity are also used. Also, 64 MB locked memory is lost to other tasks permanently, and the assumed task sizes need to be confirmed with the vendor's real figures.

Alternative concept – RTOS with no virtual memory instead of Linux with locked memory

In an RTOS (e.g. FreeRTOS, VxWorks, QNX, Zephyr) with no virtual memory, all tasks use **physical memory directly**, allocated **statically at start-up** in fixed partitions. There are no page faults and no page tables, only a possible Memory Protection Unit (MPU).

Criterion	Linux + mlock (this design)	RTOS, no virtual memory
Worst-case delay of control task	27–41 µs measured, no hard bound (unless PREEMPT_RT); depends on kernel preemption, TLB, caches	small, bounded and analysable (typically a few µs, with a deterministic scheduler and no translation faults); O(1) memory access

Criterion	Linux + mlock (this design)	RTOS, no virtual memory
Memory for ML task	1,536 MB fits in 1,792 MB (demand paging, file-backed weights can be dropped)	RTOS image and kernel are small (≈ MBs), so up to ≈ 1.9 GB could be given to ML, but it must be one static contiguous block reserved at boot, not shared or paged; no swap or lazy loading — all 1.5 GB must be physically present always
Fragmentation	none (paging)	external fragmentation if allocation is dynamic; avoided by static partitions
Effort of adding tasks	low: start another process; kernel isolates it (MMU), only admission control needed	high: re-plan the static memory map, recompile or reconfigure; no MMU isolation means a bug in the new task can corrupt the control task; porting ML frameworks (Python/TensorFlow Lite) is hard or impossible
Ecosystem	Linux drivers, networking, containers, ML libraries	small, certified, but limited libraries

Evaluation: the RTOS gives better and provable real-time behaviour, but the 1.5 GB ML workload and the vendor's plan to add more tasks favour Linux: a hybrid is common — an RTOS (or a PREEMPT_RT core with isolated CPU) for the control loop and Linux for ML and logging. **Complexity:** control-task delay is O(1) in the RTOS (no translation) and O(1) amortised plus jitter on Linux; memory: RTOS O(Σ static sizes), Linux O(Σ working sets); adding the k-th task costs O(1) on Linux and a re-plan of O(k) partitions on the RTOS.

- thms for a virtual-storage computer' (1966).